



北京邮电大学

Beijing University of Posts and Telecommunications

# 循环神经网络与Transformer

戚 琦

网络与交换技术国家重点实验室 网络智能研究中心 科研楼511

qiqi8266@bupt.edu.cn

13466759972



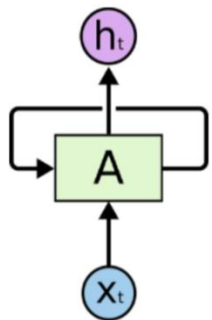


- 循环神经网络概念
- RNN的应用
- LSTM
- 注意力机制
- Transformer

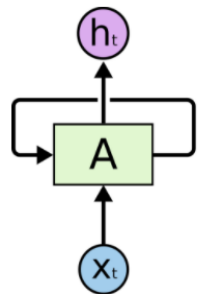




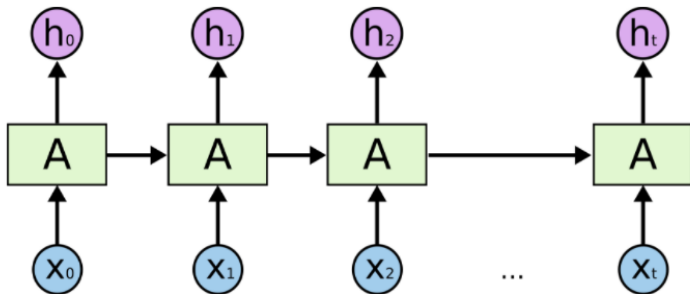
## 如何处理序列数据？



一组神经网络 **A** 接收某些输入  $x_t$ ，并输出一个值  $h_t$ 。循环允许信息从网络的一个步骤传递到下一个



=



**同一个网络的多个副本**，  
每一个都传递一个消息给  
后继者

循环神经网络与序列和列表密切相关，  
是用于此类数据的自然神经网络结构。

## One-hot编码

How to represent each word as a vector?

**1-of-N Encoding**    lexicon = {apple, bag, cat, dog, elephant}

The vector is lexicon size.

Each dimension corresponds  
to a word in the lexicon

The dimension for the word  
is 1, and others are 0

apple = [ 1   0   0   0   0 ]

bag    = [ 0   1   0   0   0 ]

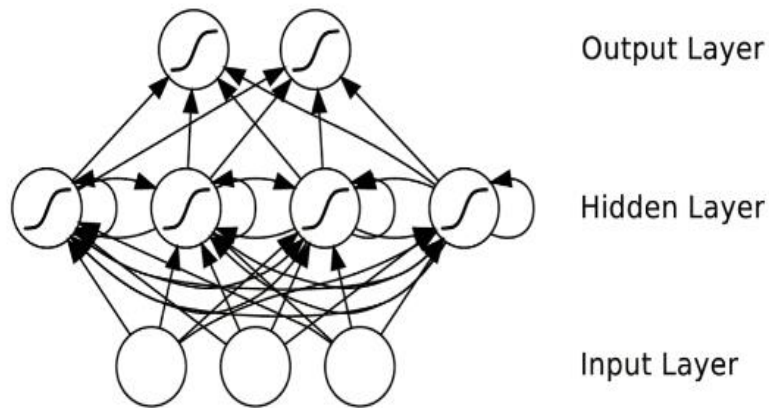
cat    = [ 0   0   1   0   0 ]

dog    = [ 0   0   0   1   0 ]

elephant = [ 0   0   0   0   1 ]



- 递归神经网络（**RNN**），是两种人工神经网络的总称
  - 一种是**时间**递归神经网络（recurrent neural network）；
  - 一种是**结构**递归神经网络（recursive neural network）；

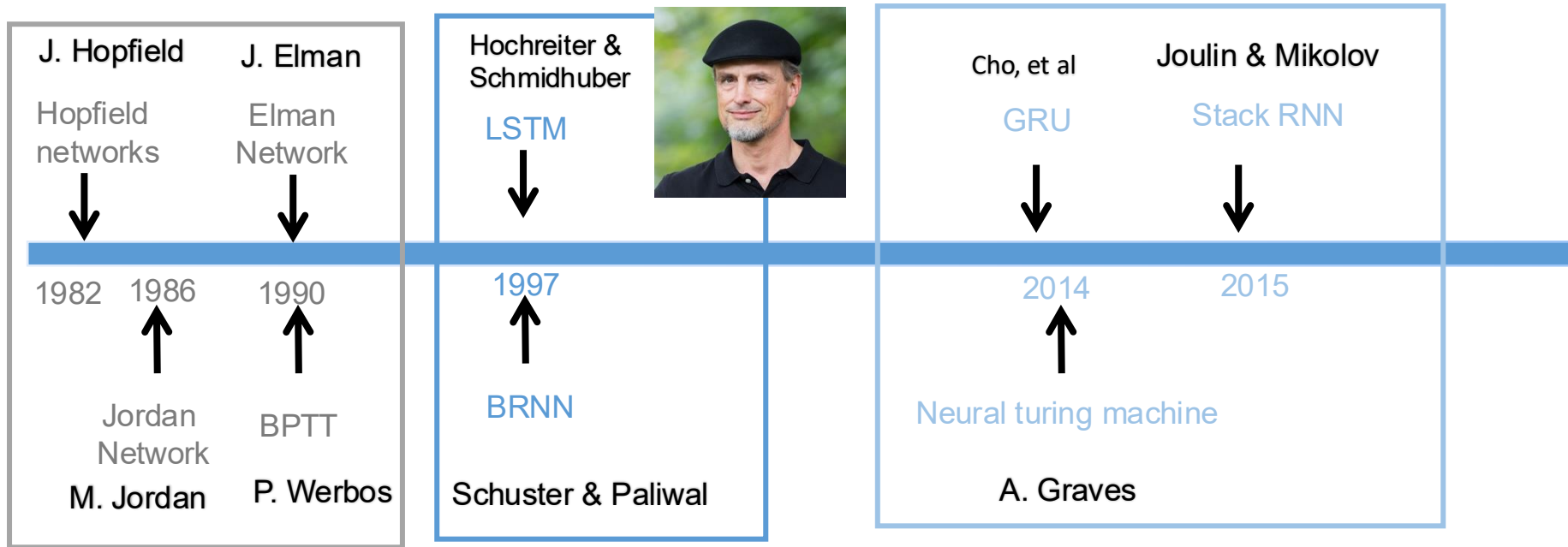


- 同层之间信息在**时序**上流动
- 吸收了HMM模型的有限序列关联的思想
- 神经网络的隐藏层结构能够更好的表达有限的观察值背后的复杂分布

主要关注的循环神经网络是时间递归神经网络



# RNN历史发展



早期（80、90年代）  
主要思想：重新使用参数和计算

中期（90-2010）  
除LSTM以外，RNN基本从主流研究中消失了。

当前（2010 - ）应用广泛：  
自然语言应用  
视频建模，手写识别，用户意图预测

2017年各类RNN形式的神经网络GRU, squeeze2squ, Attention, Transformer



- RNN是一类扩展的人工神经网络，为了对序列数据进行建模而产生。
- 针对对象：序列数据。例如文本，是字母和词汇的序列；语音，是音节的序列；视频，是图像的序列；气象观测数据，股票交易数据，设备监控数据等，都是序列数据。
- 核心思想：样本间存在顺序关系，每个样本和它之前的样本存在关联。通过神经网络在时序上的展开，能够找到样本之间的序列相关性。

$$h_t = \mathcal{H}(w_{ih}x_t + w_{hh}h_{t-1} + b_h)$$



当前时刻输入

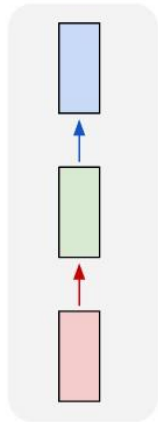


前一时刻的状态

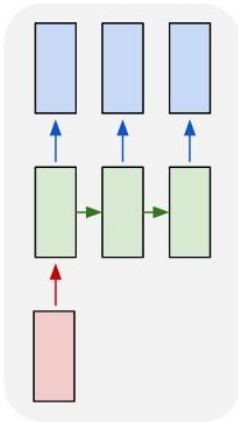
e.g. Sentiment Classification  
sequence of words  $\rightarrow$  sentiment

e.g. Video classification on frame level

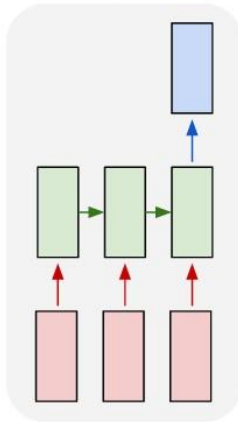
one to one



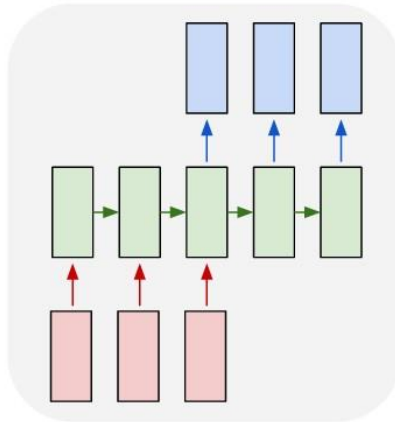
one to many



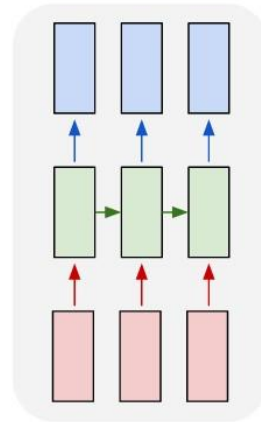
many to one



many to many



many to many



e.g. Image Captioning  
image  $\rightarrow$  sequence of words

e.g. Machine Translation  
seq of words  $\rightarrow$  seq of words

- RNN常用的激活函数是tanh和sigmoid

$$f(z) = \frac{1}{1 + \exp(-z)} \quad f(z) = \tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

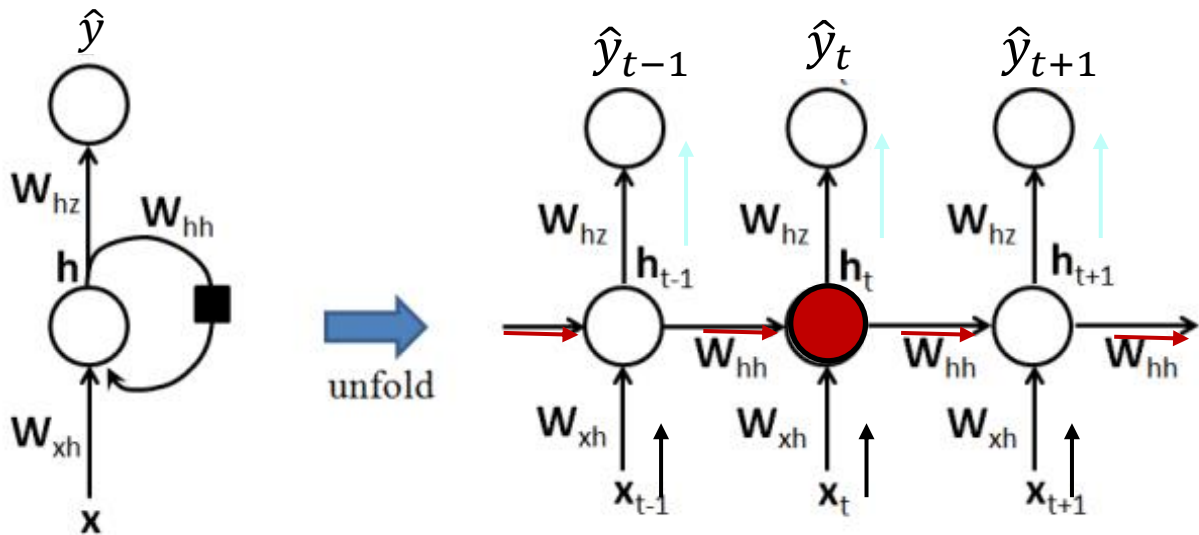
- Softmax函数：在多分类任务的输出层，将输入转化成标签的概率。

$$h_{\theta}(x^{(i)}) = \begin{bmatrix} p(y^{(i)} = 1 | x^{(i)}; \theta) \\ p(y^{(i)} = 2 | x^{(i)}; \theta) \\ \vdots \\ p(y^{(i)} = k | x^{(i)}; \theta) \end{bmatrix} = \frac{1}{\sum_{j=1}^k e^{\theta_j^T x^{(i)}}} \begin{bmatrix} e^{\theta_1^T x^{(i)}} \\ e^{\theta_2^T x^{(i)}} \\ \vdots \\ e^{\theta_k^T x^{(i)}} \end{bmatrix}$$

本质就是将一个**K**维的任意实数向量压缩（映射）成另一个**K**维的实数向量，其中向量中的每个元素取值都介于（**0**，**1**）之间。



# 时序扩展



$$h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$$

$$z_t = W_{hz}h_t + b$$

$$\hat{y}_t = \text{softmax}(z_t)$$

神经元之间的连接权重在  
时域上不变，即权值共享



BP: 定义损失函数  $E$  来表示输出  $\hat{y}$  和真实标签  $y$  的误差, 通过链式法则自顶向下求得  $E$  对网络权重的偏导。沿梯度的反方向更新权重的值, 直到  $E$  收敛。

**RNN梯度反向传播的本质: 与BP类似, 加上了时序演化。** 定义权重  $W_{xh}$ ,  $W_{hh}$ ,  $W_{hz}$

$$h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$$

$$z_t = W_{hz}h_t + b \quad \hat{y}_t = \text{softmax}(z_t)$$

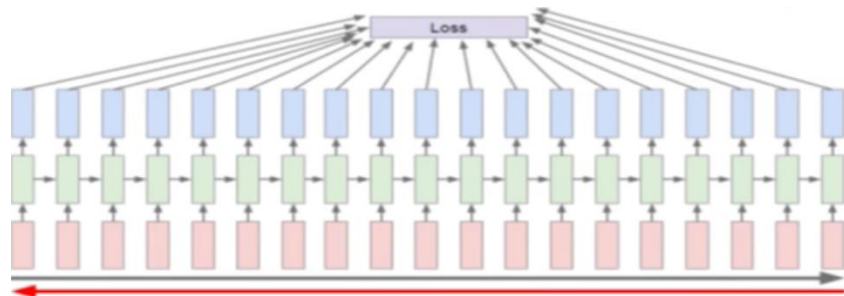
$t$ 时刻的损失函数:

$$E_t(y_t, \hat{y}_t) = -y_t \log \hat{y}_t \quad \text{一个时刻}$$

总损失函数:

$$E(y_t, \hat{y}_t) = \sum_t E_t(y_t, \hat{y}_t) = -\sum_t y_t \log \hat{y}_t \quad \text{所有时刻}$$

RNN沿着时间轴展开



**将整个序列作为一次训练, 需要对每个时刻的误差进行求和。**

求  $E$  对于  $W_{xh}$ ,  $W_{hh}$ ,  $W_{hz}$  的梯度

定义  $E$  对于  $W_{hz}$  的梯度 ( $W_{xh}$ ,  $W_{hh}$  同理):

$$\frac{\partial E}{\partial W_{hz}} = \sum_t \frac{\partial E_t}{\partial W_{hz}}$$

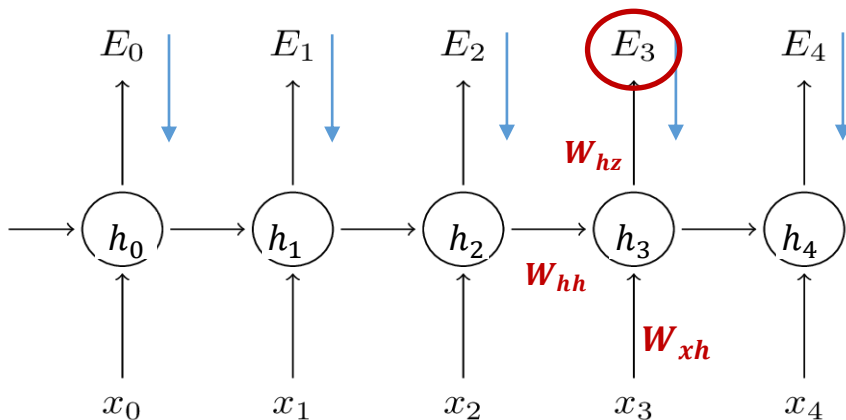
(1) 求  $E$  对于  $W_{hz}$  的梯度

先求  $E_3$  对于  $W_{hz}$  的梯度:

$$\frac{\partial E_3}{\partial W_{hz}} = \frac{\partial E_3}{\partial \hat{y}_3} \frac{\partial \hat{y}_3}{\partial W_{hz}} = \frac{\partial E_3}{\partial \hat{y}_3} \frac{\partial \hat{y}_3}{\partial z_3} \frac{\partial z_3}{\partial W_{hz}}$$

$$\frac{\partial E}{\partial W_{hz}} \quad \text{求和可得。}$$

前向计算后, 将每一个时刻  $t$  的 loss 加到一起作为总的目标函数, 逐级求导



$$h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$$

$$z_t = W_{hz}h_t + b$$

$$\hat{y}_t = \text{softmax}(z_t)$$

$$E(y_t, \hat{y}_t) = - \sum_t y_t \log \hat{y}_t$$



(2) 求 E 对于  $W_{hh}$  的梯度

注意，现在情况开始变得复杂起来。

先求  $E_3$  对于  $W_{hh}$  的梯度：

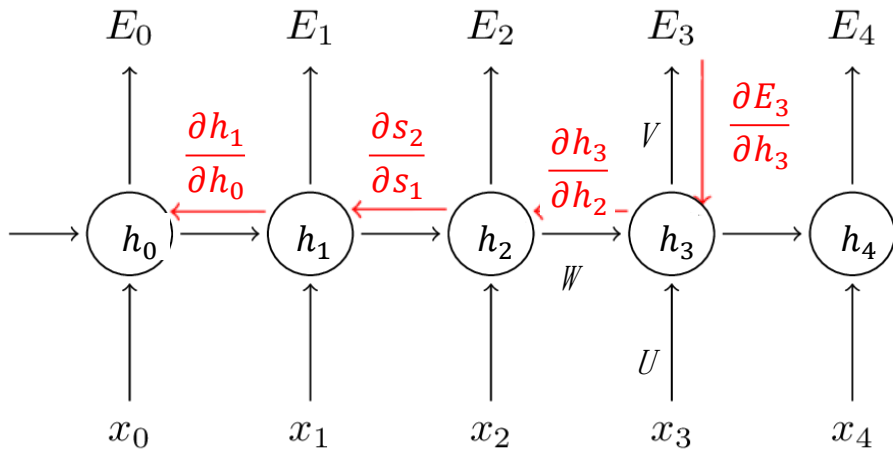
$$\frac{\partial E_3}{\partial W_{hh}} = \frac{\partial E_3}{\partial \hat{y}_3} \frac{\partial \hat{y}_3}{\partial h_3} \frac{\partial h_3}{\partial W_{hh}}$$

当我们求  $h_3$  对于  $W_{hh}$  的偏导时，注意到：

$$h_3 = \tanh(W_{xh}x_t + \mathbf{W_{hh}}h_2 + b_h)$$

其中：  $h_3$  依赖于  $h_2$  而  $h_2$  又依赖于  $h_1$  和  $W_{hh}$ ，  
依赖关系一直传递到  $t = 0$  的时刻。

因此，当我们计算对于  $W_{hh}$  的偏导数时，不能  
把  $h_2$  看作是常数项！



$$\Rightarrow \frac{\partial E_3}{\partial W_{hh}} = \sum_{t=0}^3 \frac{\partial E_3}{\partial \hat{y}_3} \frac{\partial \hat{y}_3}{\partial h_3} \frac{\partial h_3}{\partial h_t} \frac{\partial h_t}{\partial W_{hh}}$$

$$\frac{\partial h_3}{\partial h_1} = \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial h_1}$$

$$\frac{\partial E}{\partial W_{hh}} \quad \text{求和可得。}$$

# RNN的训练问题

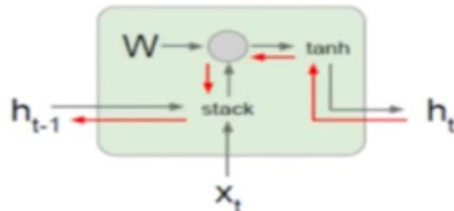
## ■ 面临的问题:

- 梯度消失问题
- 梯度爆炸问题

W提出作为误差传导的函数，例如， $h_4$ 到 $h_0$ ，W被乘一次， $h_3$ 到 $h_2$ 又乘了一次。如果W是小于1，梯度则变得很小；如果W大于1，梯度则很大

## ■ 解决方案:

- 激活函数改为Relu，大于0的部分维持1的梯度
- 把数据归一化到合适的范围，使sigmoid靠近0
- 引入改进网络结构的机制，例如LSTM、GRU。



$$\begin{aligned} h_t &= \tanh(W_{hh}h_{t-1} + W_{xh}x_t) \\ &= \tanh\left([W_{hh} \quad W_{xh}] \begin{bmatrix} h_{t-1} \\ x_t \end{bmatrix}\right) \\ &= \tanh(W \begin{bmatrix} h_{t-1} \\ x_t \end{bmatrix}) \end{aligned}$$

tanh 输出范围是 $(-1,1)$

# RNN的训练问题

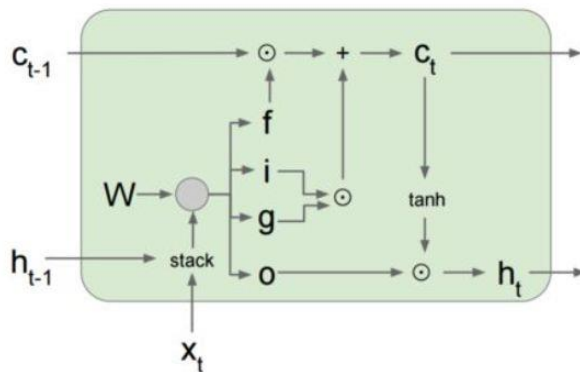
- LSTM增加了RNN单元的复杂度，使模型更具有表现力

RNN

$$h_t = \tanh(W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix})$$



LSTM



$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

$$c_t = f \odot c_{t-1} + i \odot g$$

$$h_t = o \odot \tanh(c_t)$$

相比于Vanilla RNN，LSTM的误差反向传播更方便和直接，梯度更新不存在RNN中的梯度爆炸和梯度消失现象

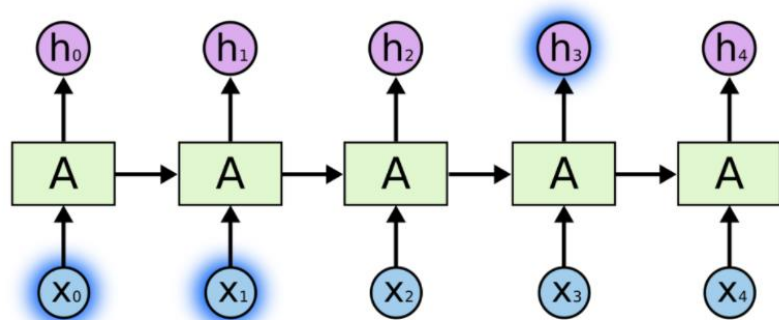


- 循环神经网络概念
- RNN的应用
- LSTM
- 注意力机制
- Transformer



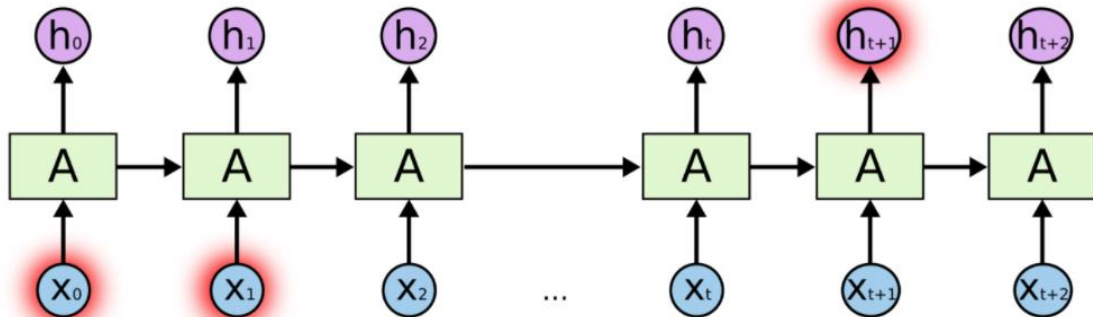


# 长短依赖



处理当前的任务，只需要查看最近的信息

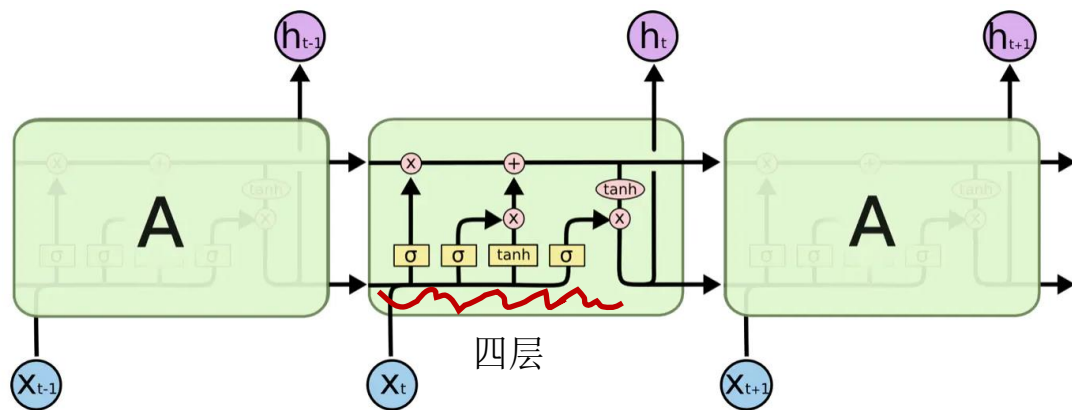
例如，考虑一种语言模型，该模型根据前面的词语来预测下一个单词，在相关信息和需要该信息的距离较近的时候，RNN能够学会去利用历史信息



预测文本中的最后一个单词 “I grew up in France... I speak fluent French.”

相关信息和需要该信息的地方的距离较远，导致RNN无法有效的利用历史信息

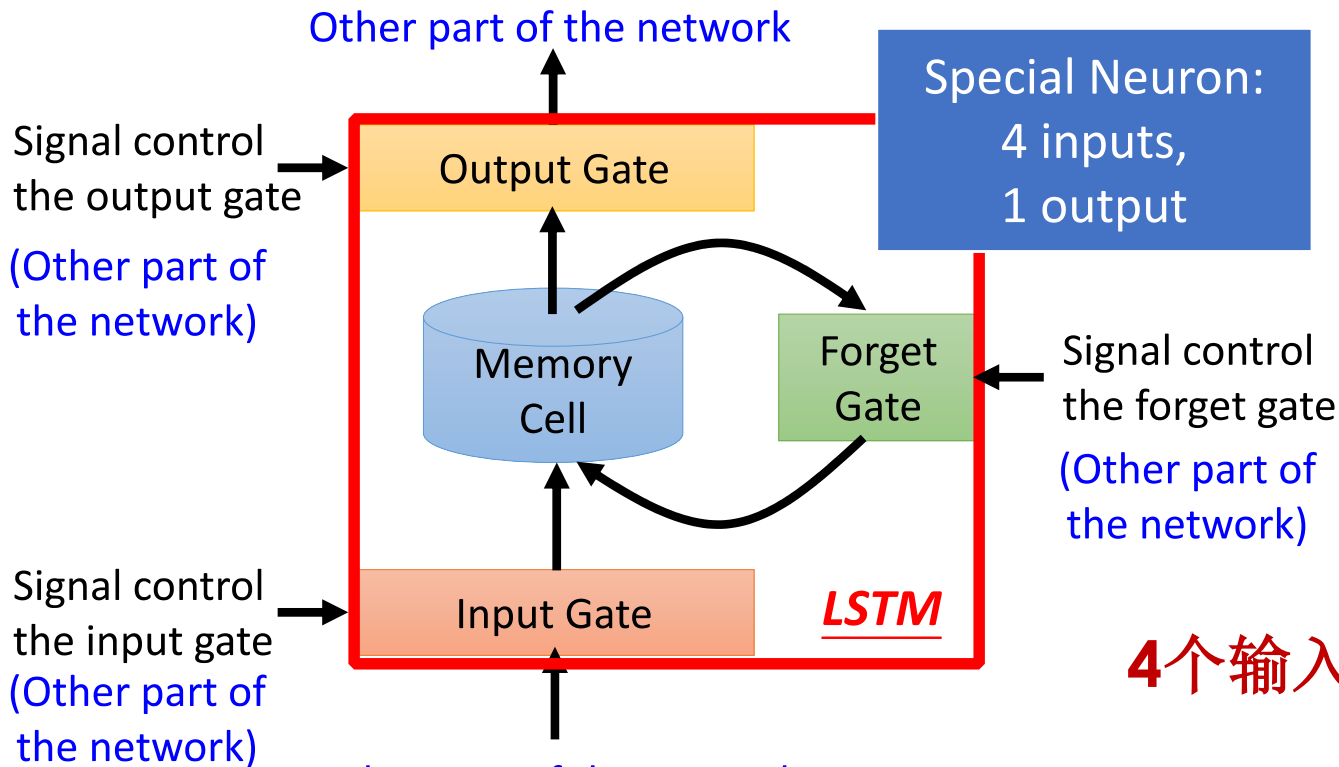
- 长短记忆神经网络——通常称作LSTM，是一种特殊的RNN，能够学习长的依赖关系。1997年，由Hochreiter&Schmidhuber提出。
- LSTM是为了避免长依赖问题而精心设计的，记住较长的历史信息实际上是RNN的默认行为，而不是努力学习的东西。



**LSTM具有链状结构，每个单元（cell）具有四个网络层**，其中操作：

- Neural Network Layer: 激活函数操作
- Pointwise Operation: 点操作
- Vector Transfer: 数据流向
- Concatenate: 表示向量的合并（concat）操作
- Copy: 向量的拷贝

# LSTM结构



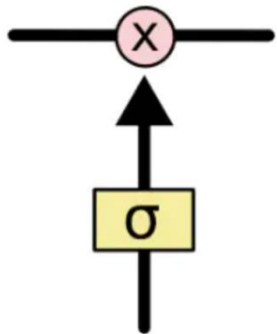
4个输入1个输出

参考李宏毅老师《机器学习》

Other part of the network  
要输入到memory中的信息



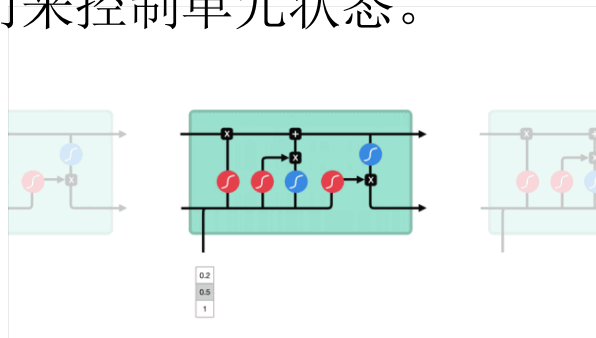
# LSTM的门



**LSTM可以作为有记忆的一个神经元！**

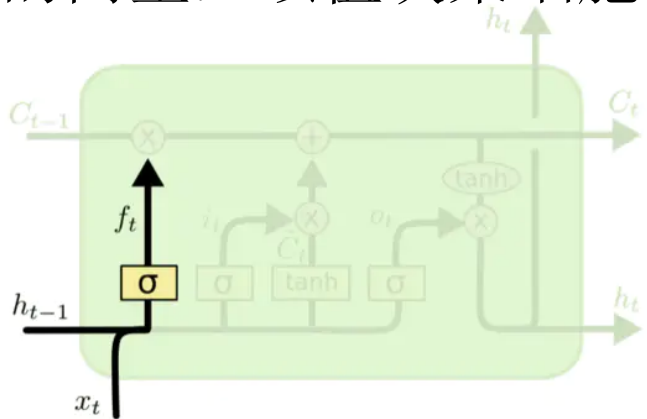
- LSTM网络能通过一种被称为**门**的结构对单元状态（cell）进行删除或者添加信息。
- 门能够有选择性的决定让哪些信息通过。
- **门的结构为一个sigmoid层和一个点乘操作的组合。**
- sigmoid层输出0-1的向量，控制多少信息能流过sigmoid层，0表示不能通过，1表示可通过

LSTM由**遗忘门**、**输入门**和**输出门**这三个门来控制单元状态。



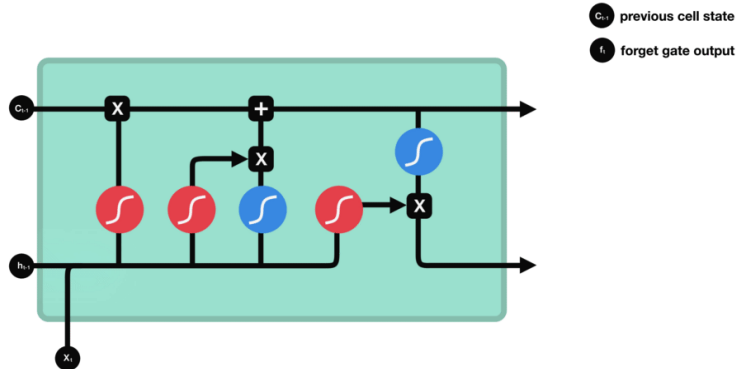


(1) 决定细胞状态需要丢弃哪些信息，通过遗忘门的sigmoid单元处理，通过查看 $h_{t-1}$ 和 $x_t$ 信息输出一个0-1之间的向量，该值决策细胞 $C_{t-1}$ 状态中哪些信息保留或丢弃



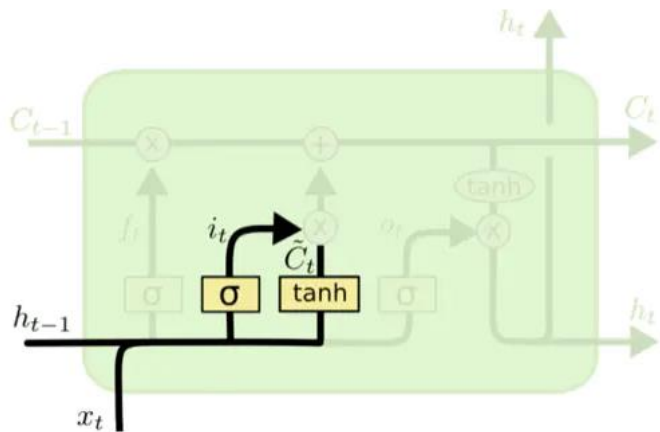
$h_{t-1}$  : 上一个cell的输出  
 $x_t$  : 当前cell的输入

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$



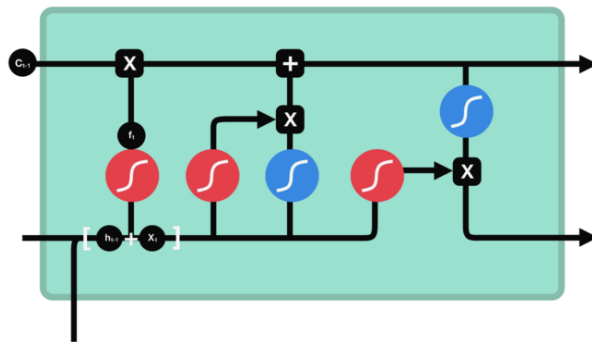


(2) 决定给细胞添加哪些新信息：①利用  $h_{t-1}$  和  $x_t$  通过输入门的操作决定更新哪些信息；②利用  $h_{t-1}$  和  $x_t$  通过一个  $\tanh$  层创造一个新的候选细胞信息  $\tilde{C}_t$ ，即更新的内容



$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

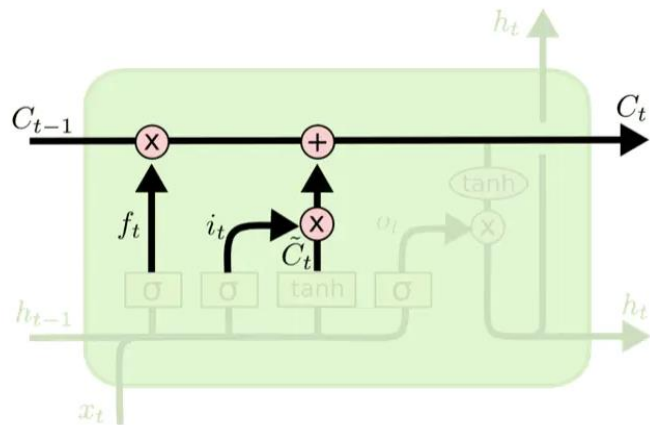
$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$



- $C_{t-1}$  previous cell state
- $f_t$  forget gate output
- $i_t$  input gate output
- $\tilde{C}_t$  candidate

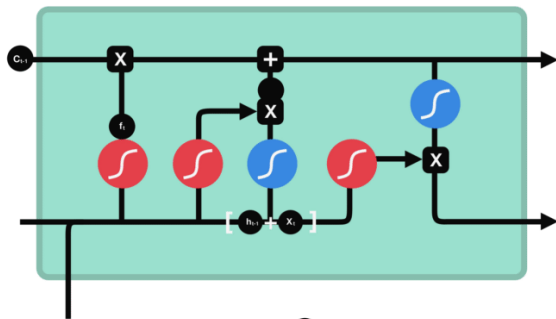
(3) 将旧的细胞信息  $C_{t-1}$  更新为新的细胞信息  $C_t$

更新规则：通过遗忘门选择忘记旧细胞信息的一部分，然后通过输入门添加候选细胞信息  $\tilde{C}_t$  的一部分得到新的细胞信息  $C_t$



$$C_t = \underbrace{f_t * C_{t-1}}_{\text{来自遗忘门}} + \underbrace{i_t * \tilde{C}_t}_{\text{来自输入门}}$$

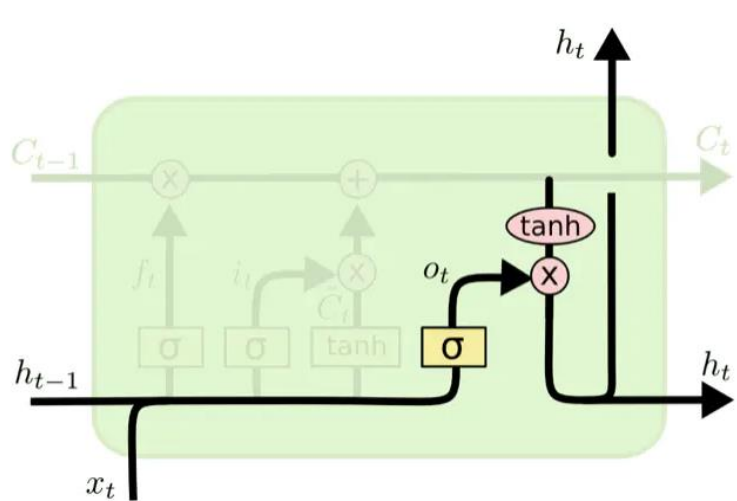
来自遗忘门    来自输入门



- $C_{t-1}$  previous cell state
- $f_t$  forget gate output
- $i_t$  input gate output
- $\tilde{C}_t$  candidate
- $C_t$  new cell state

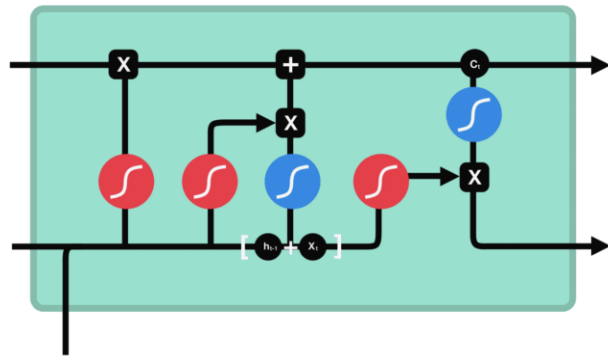
$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

(4) 更新完细胞状态后需要根据输入的 $h_{t-1}$ 和 $x_t$ 来判断输出细胞的状态特征。经过被称为输出门的sigmoid层得到判断条件，然后将细胞状态经过tanh层得到一个 $[-1,1]$ 间的向量，该向量与输入门得到的判断条件相乘即得到最终RNN单元的输出：



$$o_t = \sigma(W_o [h_{t-1}, x_t] + b_o)$$

$$h_t = \underline{o_t} * \tanh(C_t)$$

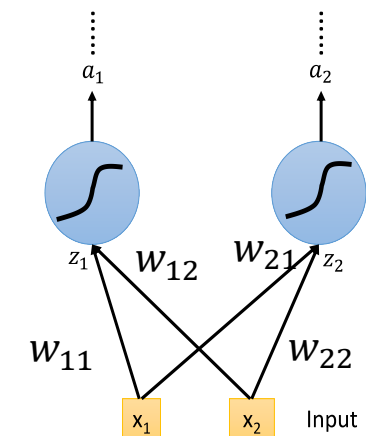


- $C_{t-1}$  previous cell state
- $f_t$  forget gate output
- $i_t$  input gate output
- $C_t$  candidate
- $C_t$  new cell state
- $o_t$  output gate output
- $h_t$  hidden state

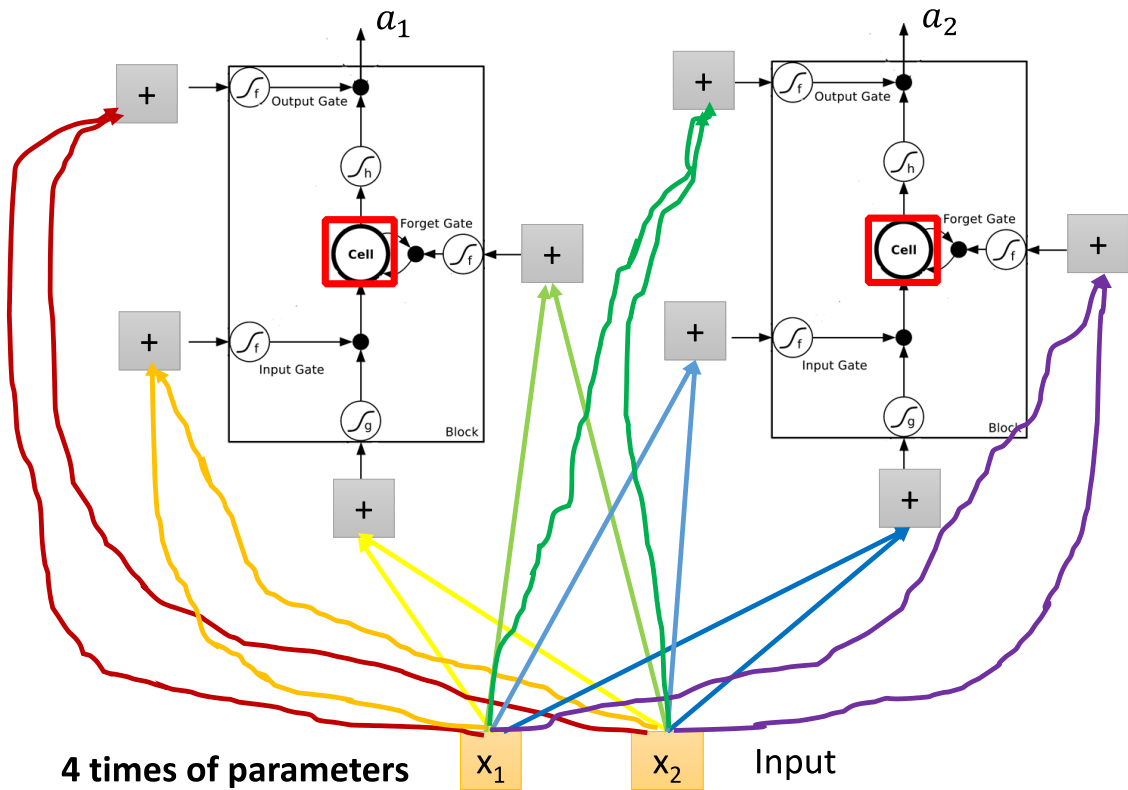
## 把NN中的神经元替换为cell

Original Network:

➤ Simply replace the neurons with LSTM



一个input一个output



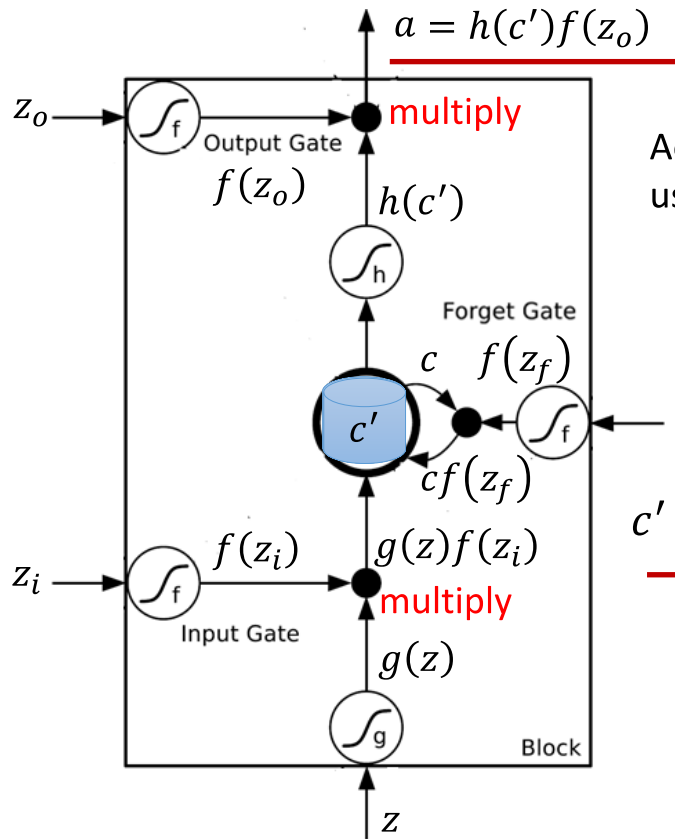
4 times of parameters

$x_1$

$x_2$

Input

将 $x_1$ 和 $x_2$ 输入到每个cell的4个门中，但是4个门的参数不同，四个不同的input一个output



Activation function  $f$  is usually a sigmoid function

Between 0 and 1

Mimic open and close gate

$$\underline{c' = g(z)f(z_i) + cf(z_f)}$$



# 门控运算举例

|       | 0 | 0 | 3 | 3 | 7 | 7 | 7  | 0 | 6 | ← Memory值      |
|-------|---|---|---|---|---|---|----|---|---|----------------|
| $x_1$ | 1 | 3 | 2 | 4 | 2 | 1 | 3  | 6 | 1 |                |
| $x_2$ | 0 | 1 | 0 | 1 | 0 | 0 | -1 | 1 | 0 | ← 输入：三维的vector |
| $x_3$ | 0 | 0 | 0 | 0 | 0 | 1 | 0  | 0 | 1 |                |
| $y$   | 0 | 0 | 0 | 0 | 0 | 7 | 0  | 0 | 6 | ← 输出           |

When  $x_2 = 1$ , add the numbers of  $x_1$  into the memory

When  $x_2 = -1$ , reset the memory

When  $x_3 = 1$ , output the number in the memory.

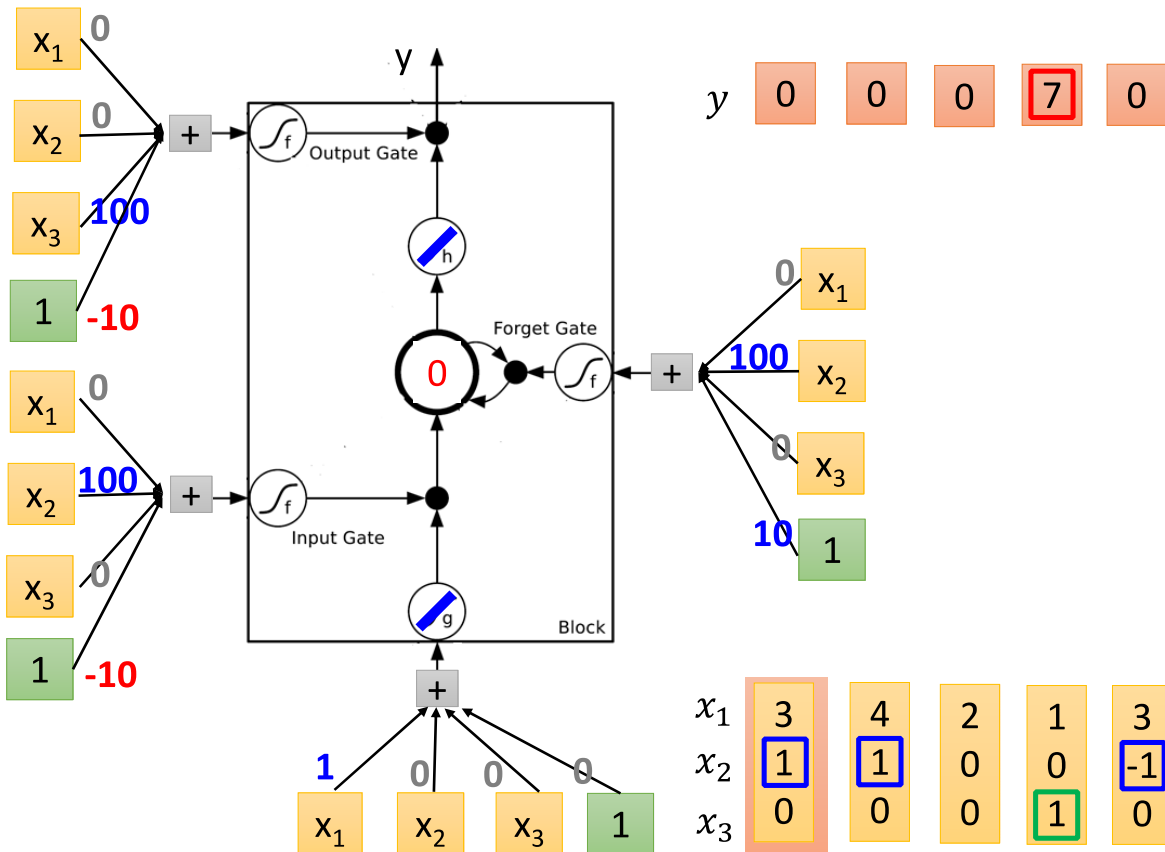


对门的操作相当于这样

假设存到memory里面的值是0，  
第二个 $x_2=1$ 时，3写入；  
第四个 $x_2=1$ ，4会写入，得到7；  
第六个 $x_3=1$ ，这时候7会被输出；  
第七个 $x_2=1$ ，memory里值为0；  
第八个 $x_2=1$ ，6写入，因为 $x_3=1$ ，  
将6输出。



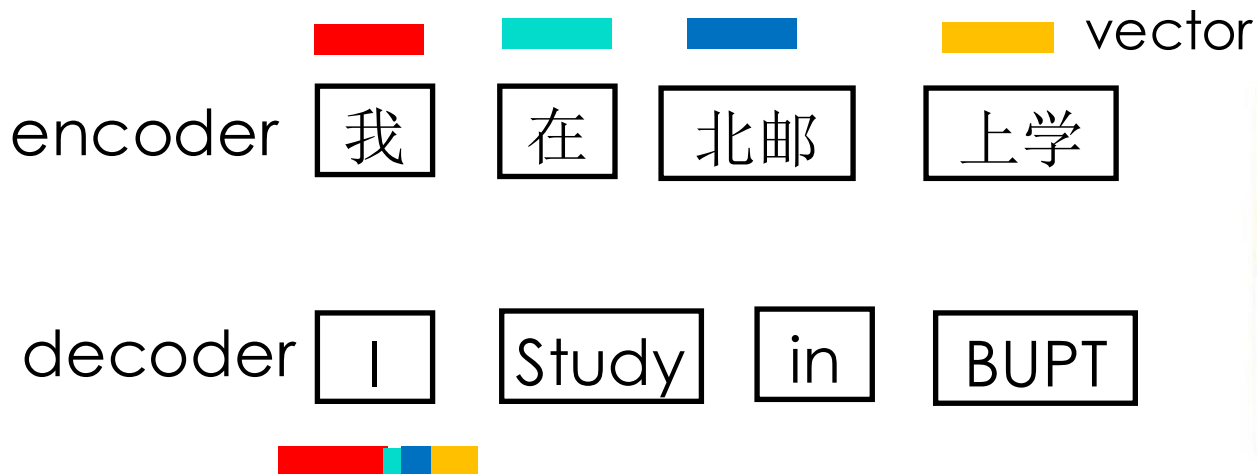
# LSTM运算举例





- 循环神经网络概念
- RNN的应用
- LSTM
- 注意力机制
- Transformer
- 预训练模型

- 对于输入的数据，关注点是什么？
- 如何才能让模型关注到有价值的内容？



**动机1：** 一个词在不同语境中有不同的含义，应该适应上下文

- Attention机制最早是应用于图像领域
- 2014年google mind团队发表论文《Recurrent Models of Visual Attention》用于图像任务，代替CNN，解决卷积计算量大的问题
- Bahdanau等人在论文《Neural Machine Translation by Jointly Learning to Align and Translate》中，使用类似Attention的机制在机器翻译任务上将翻译和对齐同时进行，他们的工作算是第一个将Attention机制应用到NLP领域中
- 2017年，Google机器翻译团队发表的《Attention is all you need》中大量使用了自注意力（self-attention）机制来学习文本表示
- 目前已经非常广泛应用于**NLP**、**CV**以及其他机器学习任务



## Recurrent Models of Visual Attention

Volodymyr Mnih Nicolas Heess Alex Graves Koray Kavukcuoglu  
Google DeepMind

{vmnih, heess, graves, korayk}@google.com

### Abstract

Applying convolutional neural networks to large images is computationally expensive because the amount of computation scales linearly with the number of image pixels. We present a novel recurrent neural network model that is capable of extracting information from an image or video by adaptively selecting a sequence of regions or locations and only processing the selected regions at high resolution. Like convolutional neural networks, the proposed model has a degree of translation invariance built-in, but the amount of computation it performs can be controlled independently of the input image size. While the model is non-differentiable, it can be trained using reinforcement learning methods to learn task-specific policies. We evaluate our model on several image classification tasks, where it significantly outperforms a convolutional neural network baseline on cluttered images, and on a dynamic visual control problem, where it learns to track a simple object without an explicit training signal for doing so.

Recurrent models typically factor computation along the symbol positions of the input and output sequences. Aligning the positions to steps in computation time, they generate a sequence of hidden states  $h_t$ , as a function of the previous hidden state  $h_{t-1}$  and the input for position  $t$ . This inherently sequential nature precludes parallelization within training examples, which becomes critical at longer sequence lengths, as memory constraints limit batching across examples. Recent work has achieved significant improvements in computational efficiency through factorization tricks [21] and conditional computation [32], while also improving model performance in case of the latter. The fundamental constraint of sequential computation, however, remains.

Attention mechanisms have become an integral part of compelling sequence modeling and transduction models in various tasks, allowing modeling of dependencies without regard to their distance in the input or output sequences [2, 19]. In all but a few cases [27], however, such attention mechanisms are used in conjunction with a recurrent network.

In this work we propose the Transformer, a model architecture eschewing recurrence and instead relying entirely on an attention mechanism to draw global dependencies between input and output. The Transformer allows for significantly more parallelization and can reach a new state of the art in translation quality after being trained for as little as twelve hours on eight P100 GPUs.

## Attention Is All You Need

Ashish Vaswani\* Noam Shazeer\* Niki Parmar\* Jakob Uszkoreit\*  
Google Brain Google Brain Google Research Google Research  
avaswani@google.com noam@google.com nikip@google.com usz@google.com

Llion Jones\* Aidan N. Gomez\*† Lukasz Kaiser\*  
Google Research University of Toronto Google Brain  
llion@google.com aidan@cs.toronto.edu lukasz.kaiser@google.com

Illia Polosukhin\*†  
illia.polosukhin@gmail.com

## References

- [1] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.
- [2] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. *CoRR*, abs/1409.0473, 2014.
- [27] Ankur Parikh, Oscar Täckström, Dipanjan Das, and Jakob Uszkoreit. A decomposable attention model. In *Empirical Methods in Natural Language Processing*, 2016.
- [28] Romain Paulus, Caiming Xiong, and Richard Socher. A deep reinforced model for abstractive summarization. *arXiv preprint arXiv:1705.04304*, 2017.

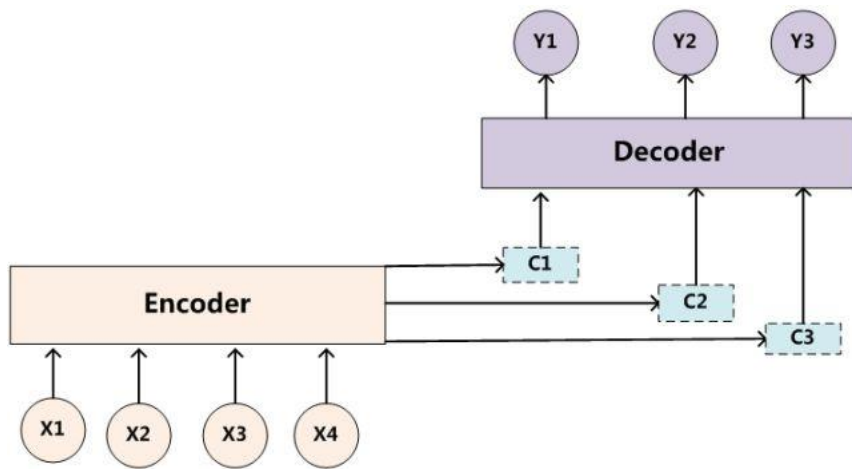


# Attention的理解

- Attention机制其实就是一系列注意力分配系数，也就是**一系列权重**参数。

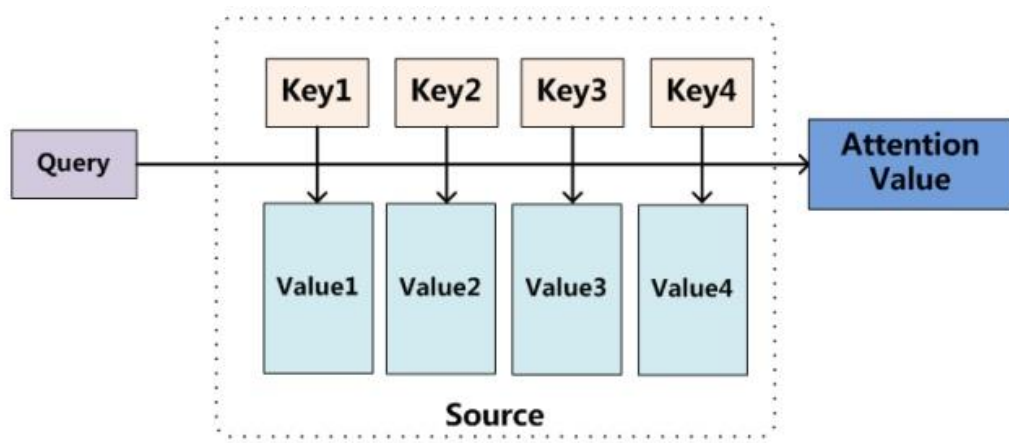
注意力模型就是要**从序列中学习**到**每一个元素的重要程度**，  
然后按重要程度将元素合并

Encoder-decoder例子



# Attention机制

- 既然Attention是一组注意力分配系数的，那么它怎样实现？这里要提出一个函数叫做**Attention函数**，用来得到Attention value的。比较主流的Attention框架如下：

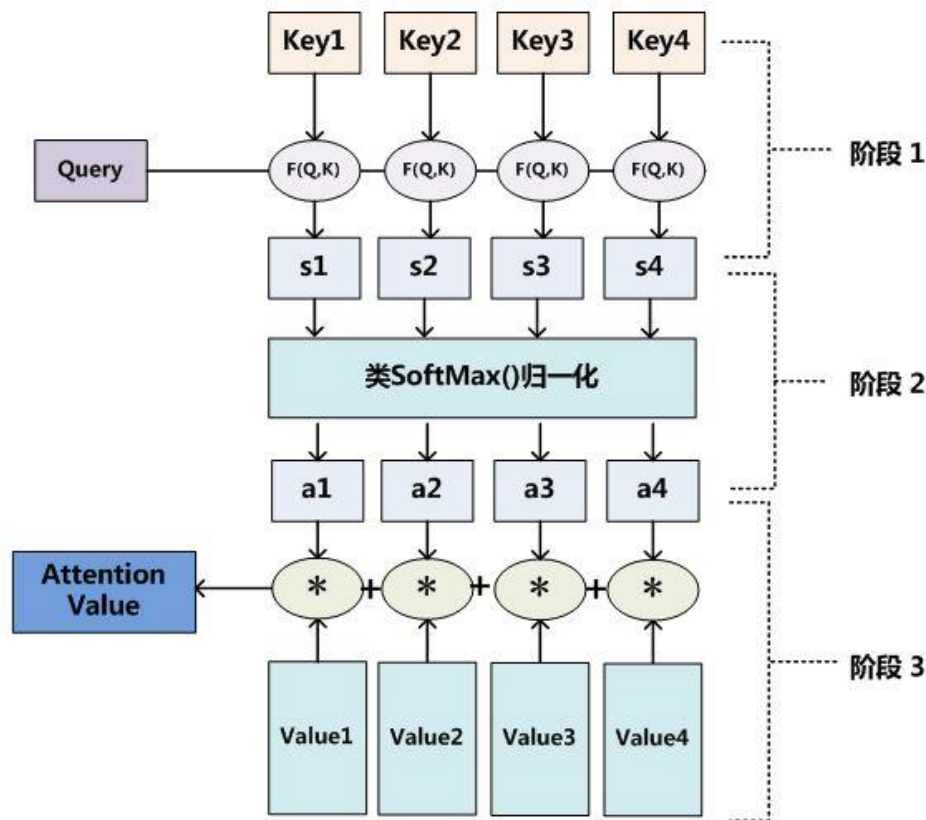


是一个查询(query)到一系列键值(key-value)对的映射。

计算**query**与**key**的相似度，决定取出来value，相似度决定了取出来值的重要程度，按重要程度合并value值得到attention值



# Attention机制



Attention函数共有三步完成得到 attention value

- **Q与K进行相似度计算得到权值**-> $s$
- 对上部权值归一化-> $a$
- **用归一化的权值与V加权求和**, 此时加权求和的结果为注意力值。

在自然语言任务中，往往K和V是相同的。这时计算出的attention value是一个向量，代表序列元素  $x_j$  的编码向量，包含了元素  $x_j$  的上下文关系。



# q与k的权重计算函数

- 多层感知机方法：先将query和key进行拼接，然后接一个激活函数为tanh的全连接层，再与一个网络定义的权重矩阵做乘积，大规模数据有优势

$$a(q, k) = w_2^T \tanh(W_1 [q; k])$$

- Bilinear方法：通过一个权重矩阵直接建立q和k的关系映射，计算速度较快

$$a(q, k) = q^T W k$$

- **Dot Product（点积）方法**：直接建立q和k的关系映射（点积是计算两个矩阵复杂度的方法之一），计算速度更快，且不需要参数，降低了模型的复杂度，但是需要q和k的维度要相同

$$a(q, k) = q^T k$$

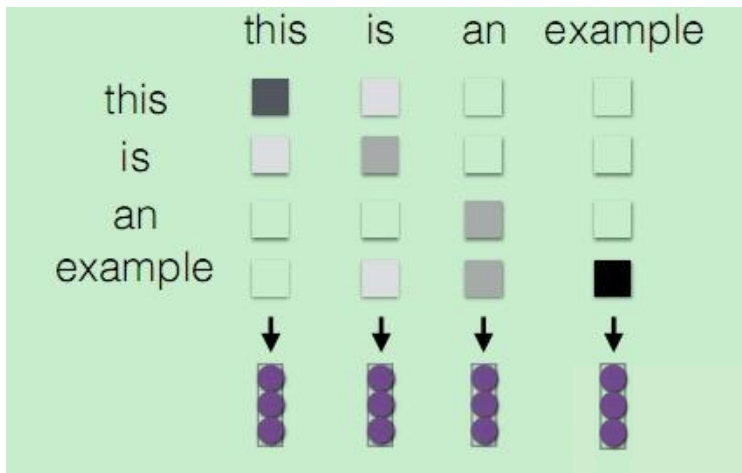
- **scaled-dot Product**：防止数据上溢，对其进行scaling  $a(q, k) = \frac{q^T k}{\sqrt{|k|}}$





# self-attention

- 关于attention有很多应用，在非seq2seq任务中，比如文本分类，或者其他分类问题，会通过self-attention来使用attention



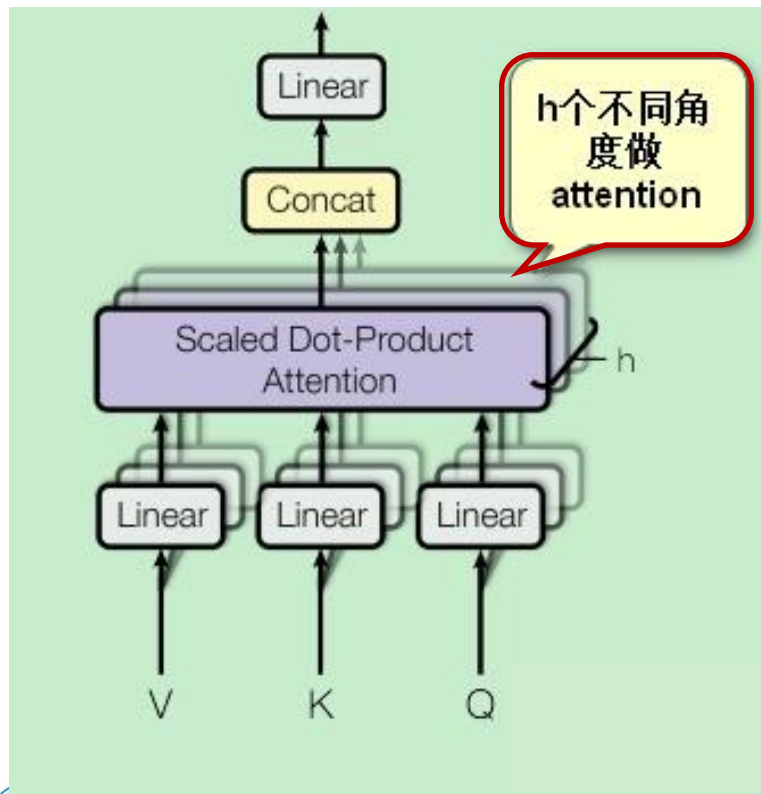
- 1、首先建立句子序列中的每个词与句子其他词k的注意力权重
- 2、然后将注意力权重向量进行softmax归一化，并与句子序列所有时刻的信息（词向量或者RNN的隐藏状态）进行线性加权。

句子中的每个词都能与句子中任意距离的其他词建立一个敏感的关系，在一定程度上提升了CNN和RNN对于长距离语义依赖建模能力的上限。

# Multi-head attention

Attention is all you need

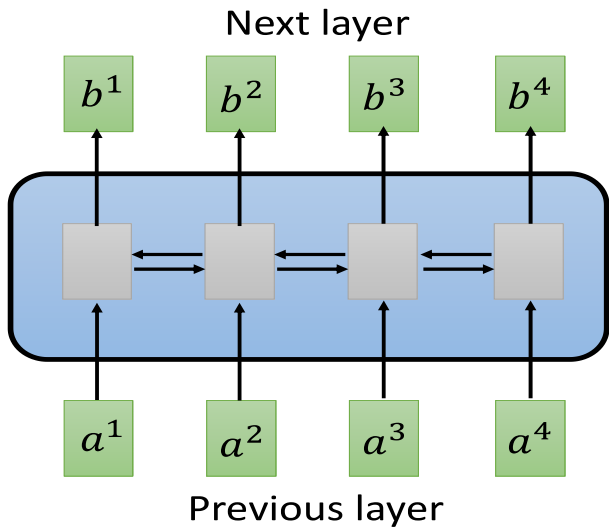
$$\text{multihead}(Q, K, V) = \text{concat}(\text{head}_1, \text{head}_2, \dots, \text{head}_h) W^O$$
$$\text{head}_i = \text{attention}(QW^Q, KW^K, VW^V)$$



设计 $h$ 种不同的  $(W_i^Q, W_i^K, W_i^V)$  权重矩阵对，然后做基本的attention操作前，将query、key和value分别用上述权重对做线性变换，然后再计算得到 $h$ 个不同角度的attention权重 $head_i$ ，将这些 $head_i$ 按列拼接后，再与一个新的权重矩阵 $W^O$ 做线性变换，得到最终的attention输出。

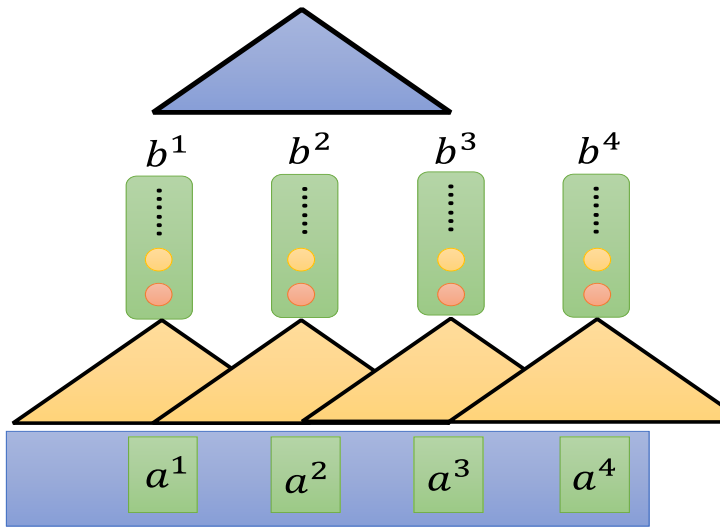
# 另一个角度：训练的效率问题

Sequence



Hard to parallel

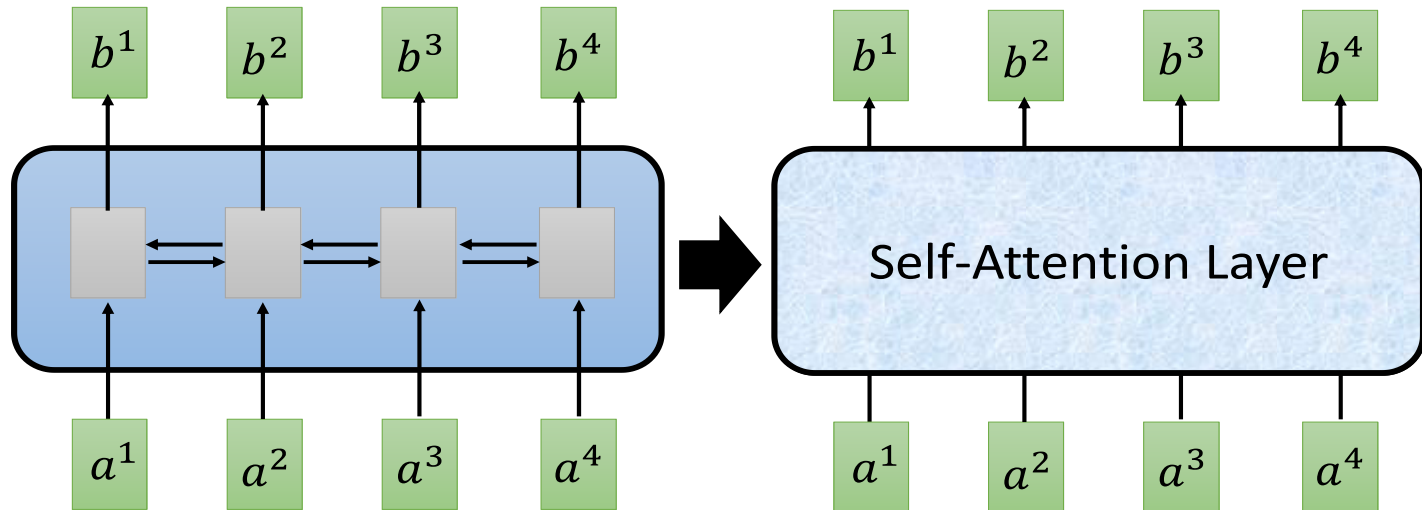
Filters in higher layer can consider longer sequence



Using CNN to replace RNN  
(CNN can parallel)

RNN串行无法堆叠很多，CNN可以代替RNN并行训练

# 利用注意力机制



You can try to replace any thing that has been done by RNN with self-attention.

动机2：注意力机制可以同时计算，可以取代RNN



## Self-attention

<https://arxiv.org/abs/1706.03762>



$q$ : query (to match others)

$$q^i = W^q a^i$$

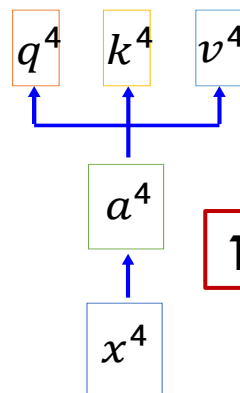
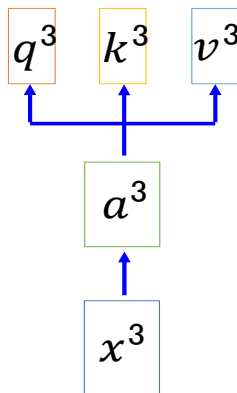
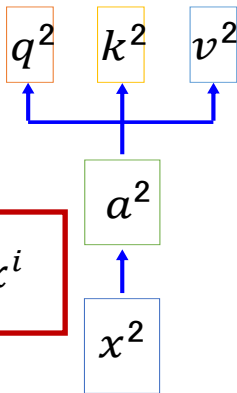
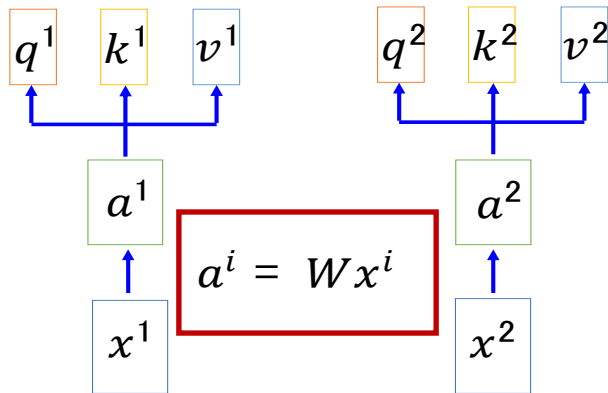
$k$ : key (to be matched)

$$k^i = W^k a^i$$

$v$ : information to be extracted

$$v^i = W^v a^i$$

2. 利用 $a$ 计算 $q$ 、 $k$ 、 $v$



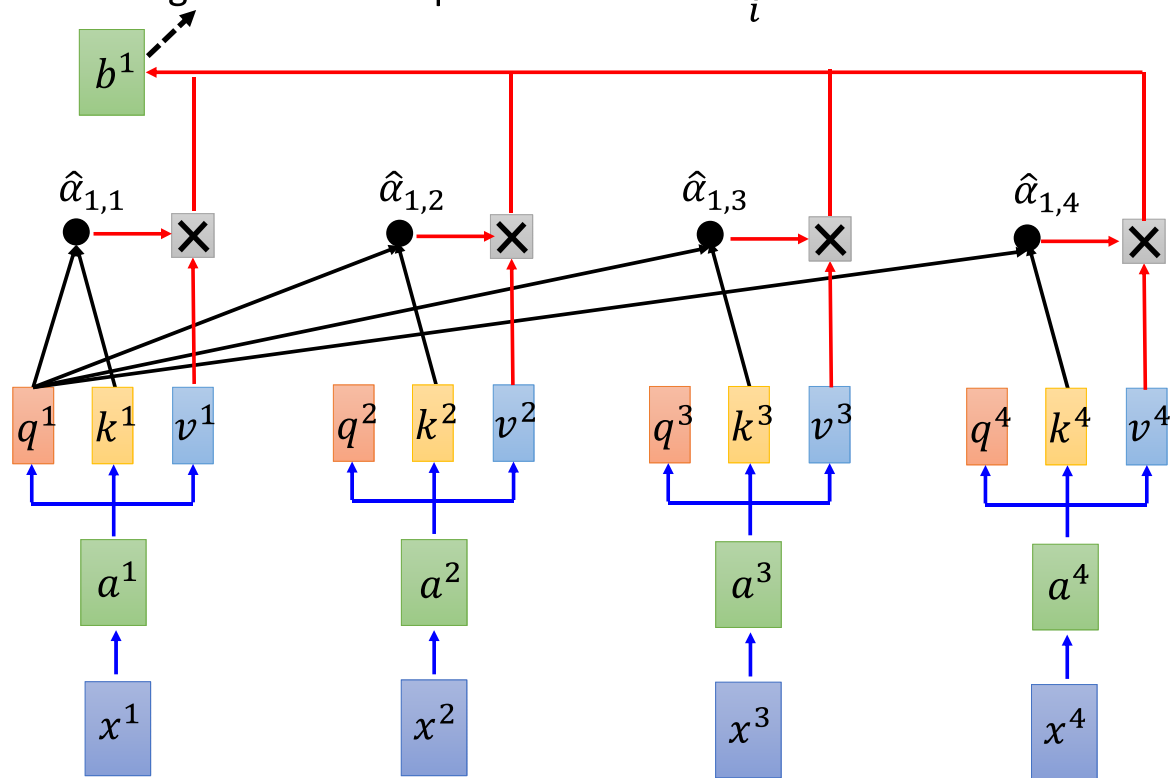
1. 给每个 $x$ 计算 $a$

# 自注意力机制

## Self-attention

Considering the whole sequence

$$b^1 = \sum_i \hat{\alpha}_{1,i} v^i$$



$\hat{\alpha}_{1,i}$  与  $v$  相乘，再把所有的相加得到 attention 值

经过 softmax 层得到  $\hat{\alpha}_{1,i}$

$$\hat{\alpha}_{1,i} = \exp(\alpha_{1,i}) / \sum_j \exp(\alpha_{1,j})$$

softmax

每个  $x$  的  $q$  与所有  $x$  的  $k$  计算  $\alpha$  (alpha)

$$\alpha_{1,i} = \underbrace{q^1 \cdot k^i}_{\text{dot product}} / \sqrt{d}$$



- 循环神经网络概念
- RNN的应用
- LSTM
- 注意力机制
- Transformer
- 预训练模型



# Attention is All You Need

视觉方向

**Vit**

Swin Transformer

时间序列

Informer

AutoFormer

RL训练

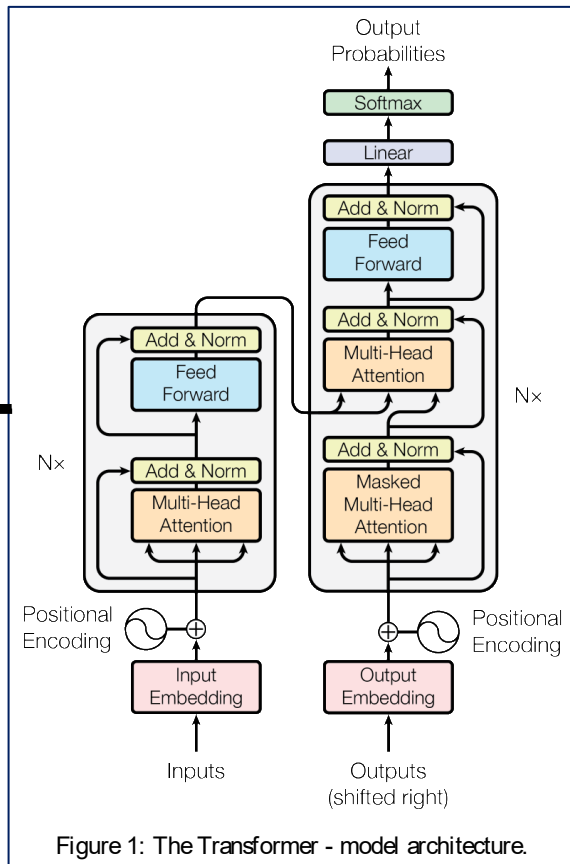


Figure 1: The Transformer - model architecture.

大语言模型

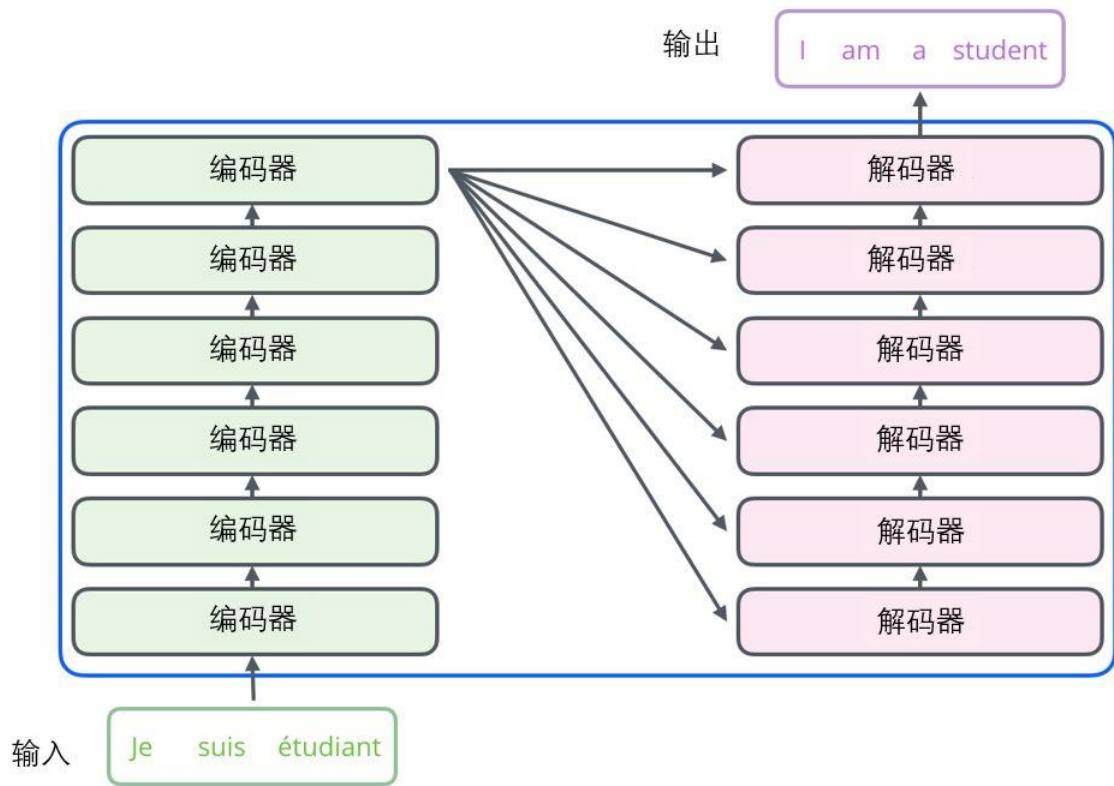
BERT

GPTx

多模态大模型



# Transformer结构



**编码组件**部分由一堆编码器（encoder）构成。

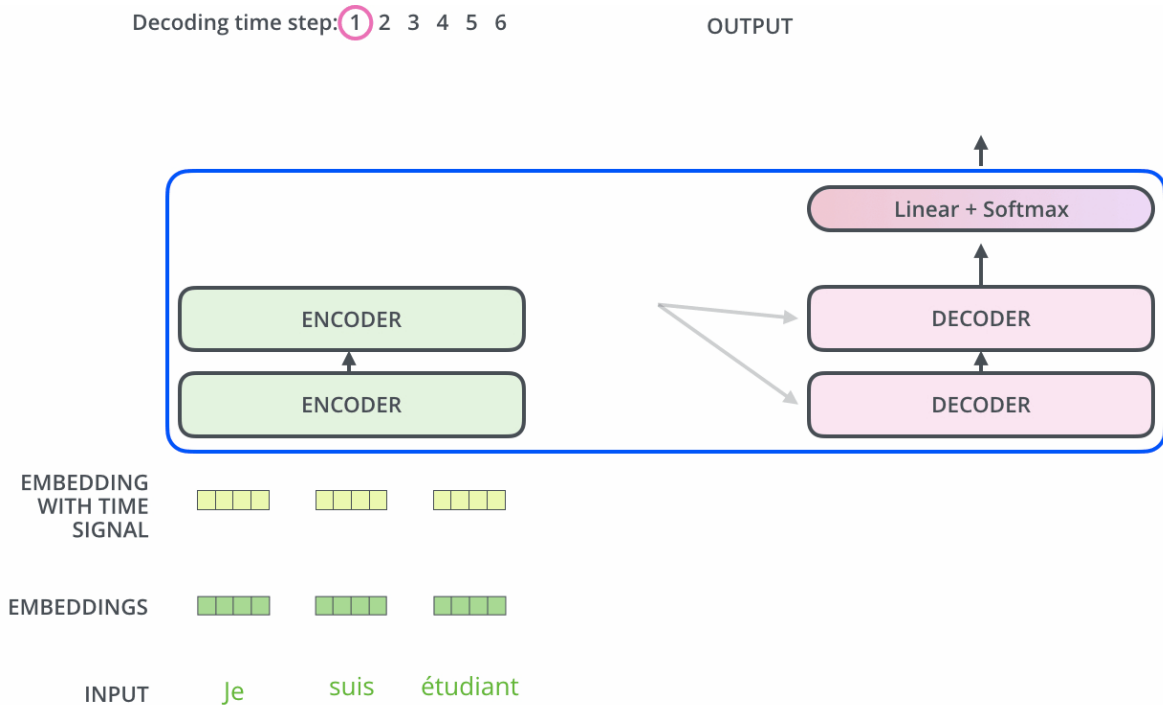
**解码组件**部分也是由相同数量（与编码器对应）的解码器（decoder）组成的。

所有编码器在结构上都是相同的，但它们没有共享参数。

每个解码器都可以分解成两个子层。



# 编码器工作流程



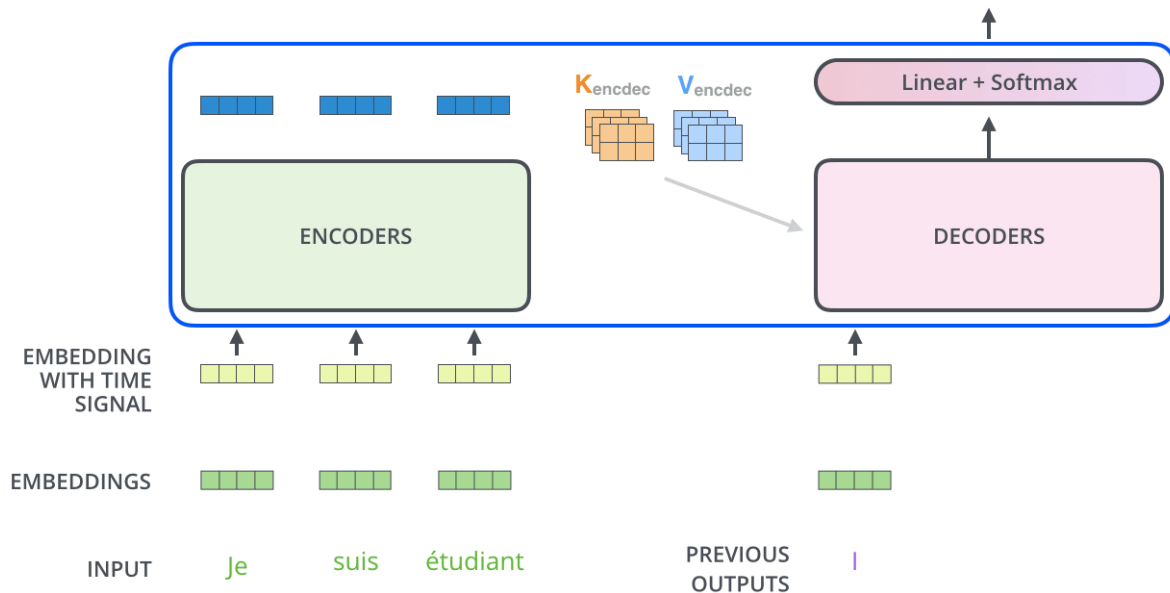
Encoder是双向的，  
每个单词都能同时  
“看到”前文和后文，  
一次性生成所有输出



# 解码器工作流程

Decoding time step: 1 2 3 4 5 6

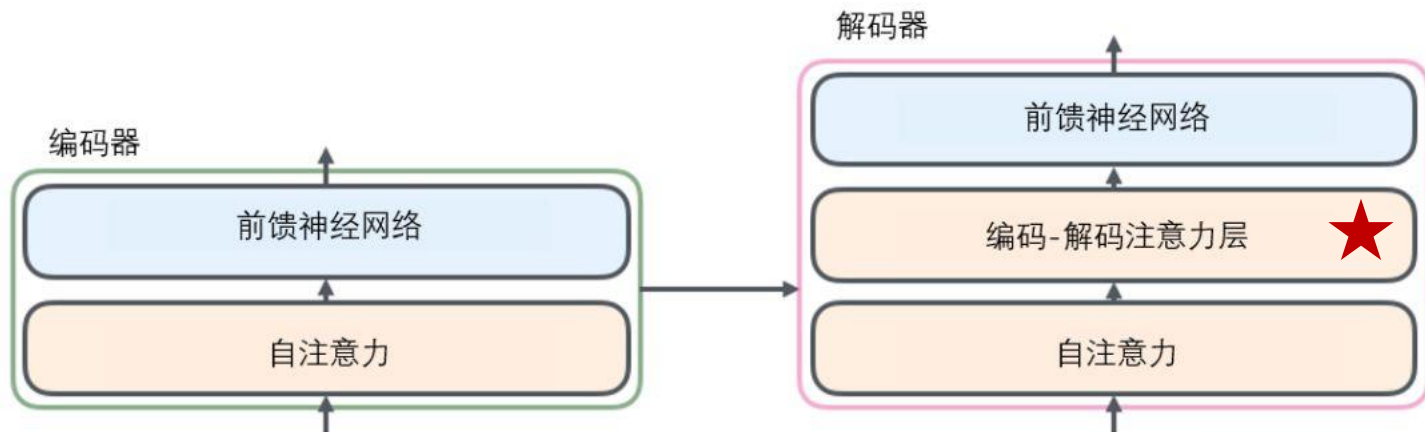
OUTPUT |



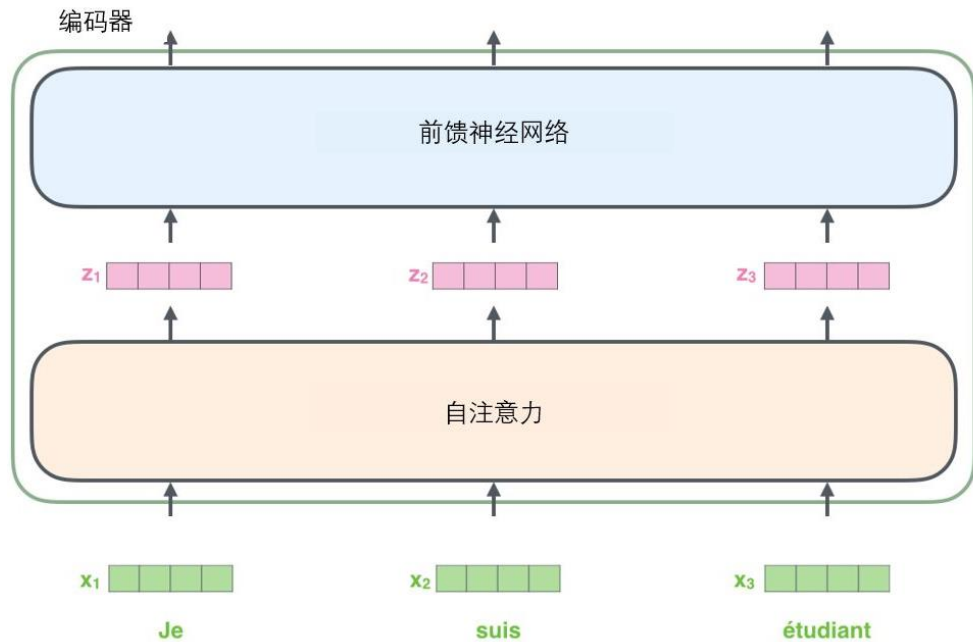
Decoder是单向的，  
生成式运行，每次  
只能运行通过前半  
句生成一个单词



# Transformer结构



- 从编码器输入的句子首先会经过一个自注意力（self-attention）层，这层帮助编码器在对每个单词编码时关注输入句子的其他单词。
- 自注意力层的输出会传递到前馈（feed-forward）神经网络中。每个位置的单词对应的前馈神经网络完全一样。
- 解码器中也有编码器的自注意力（self-attention）层和前馈（feed-forward）层。除此之外，这两个层之间还有一个注意力层，让解码层接收编码层提取的语义信息，关注输入句子的相关部分（和seq2seq模型的注意力作用相似）。



每个单词都被嵌入为512维的向量

单词的 Embedding 有很多方式可以获取，例如可以采用 Word2Vec、Glove 等算法预训练得到，也可以在 Transformer 中训练得到。

词嵌入过程只发生在最底层的编码器中。所有的编码器都有一个相同的特点，即它们接收一个向量列表，列表中的每个向量大小为512维。

在底层（最开始）编码器中它就是词向量，但是在其他编码器中，它就是下一层编码器的输出（也是一个向量列表）。

向量列表大小是可设置的超参数，一般是训练集中最长句子的长度。

将输入序列进行词嵌入之后，每个单词都会流经编码器中的两个子层。

## 1. 预处理：从文字到数字 (Tokenization)

在进入 Transformer 之前，原始文本首先要经过分词器 (Tokenizer) 。

- **分词**：将句子拆分为 Token（可能是词、子词或字符）。例如，"Learning is fun" 可能会被拆分为 ["Learning", "is", "fun"]。
- **映射 ID**：每一个 Token 都会根据预先训练好的词表 (Vocabulary)，转换成一个唯一的整数索引 (ID)。
  - "Learning" → 15496
  - "is" → 318
  - "fun" → 1254

## 2. 核心逻辑：查找表 (Look-up Table)

进入模型后，词嵌入层实际上是一个巨大的矩阵，我们通常称之为  $W_e$ 。

- 矩阵维度：  $V \times d_{model}$ 
  - $V$ ：词表大小（如 GPT-3 约为 50,257，Llama 3 约为 128,256）。
  - $d_{model}$ ：嵌入向量的维度（如 GPT-3 是 12288，Llama 2-7B 是 4096）。
- 查找操作：词嵌入不是通过复杂的计算得到的，而是通过 **ID 索引** 直接从矩阵中提取对应的行。
  - 如果 "Learning" 的 ID 是 15496，模型就会取出矩阵  $W_e$  的第 15496 行。这一行就是一个长度为  $d_{model}$  的实数向量。

词嵌入的本质是语义降维与连续化。

1. **特征表达**: 一个词不再是一个孤立的符号, 而是由数千个维度共同描述的特征集合。例如, 向量中某些维度可能隐式代表了“词性”、“情感极性”或“实体类别”。
2. **计算相似度**: 变成向量后, 模型可以使用余弦相似度等数学方法, 计算两个词之间的关系。

在高维空间里, "猫" 和 "狗" 的向量距离, 通常比 "猫" 和 "手机" 的距离更近。

| 特性   | 训练阶段 (Training)                 | 推理阶段 (Inference)             |
|------|---------------------------------|------------------------------|
| 权重状态 | 矩阵 $W_e$ 的数值是随机初始化的, 随梯度下降不断更新。 | 矩阵 $W_e$ 是固定的, 直接调用已学到的语义特征。 |
| 目标   | 学习如何让意思相近的词在向量空间里靠得更近。          | 将输入的词精准映射到已有的语义空间。           |
| 并行性  | 整个句子 (Sequence) 一次性嵌入。          | 逐个 Token 进行 (如果是自回归生成)。      |





# 例子

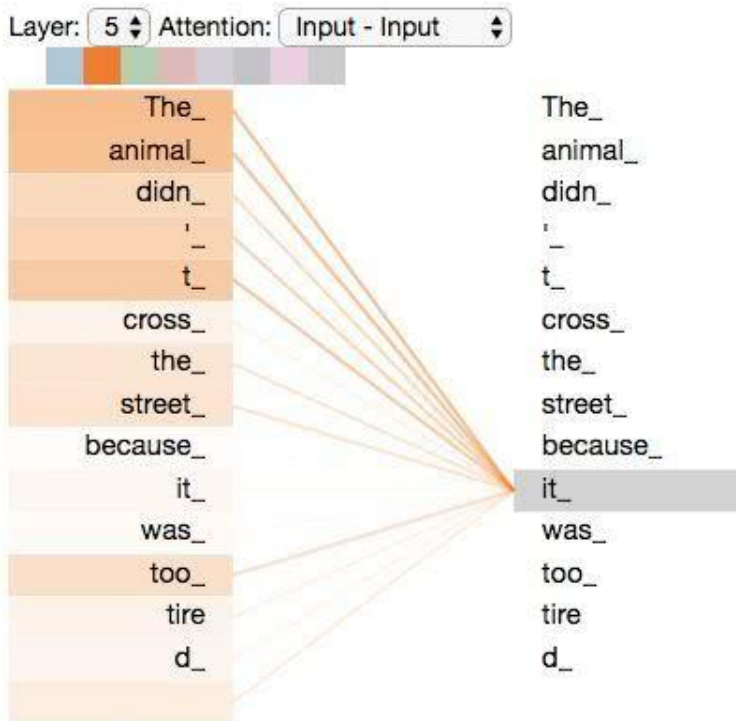
句子翻译：

The animal didn't cross the street  
because **it** was too tired.

(The animal didn't cross the street  
because **it** was too narrow.)

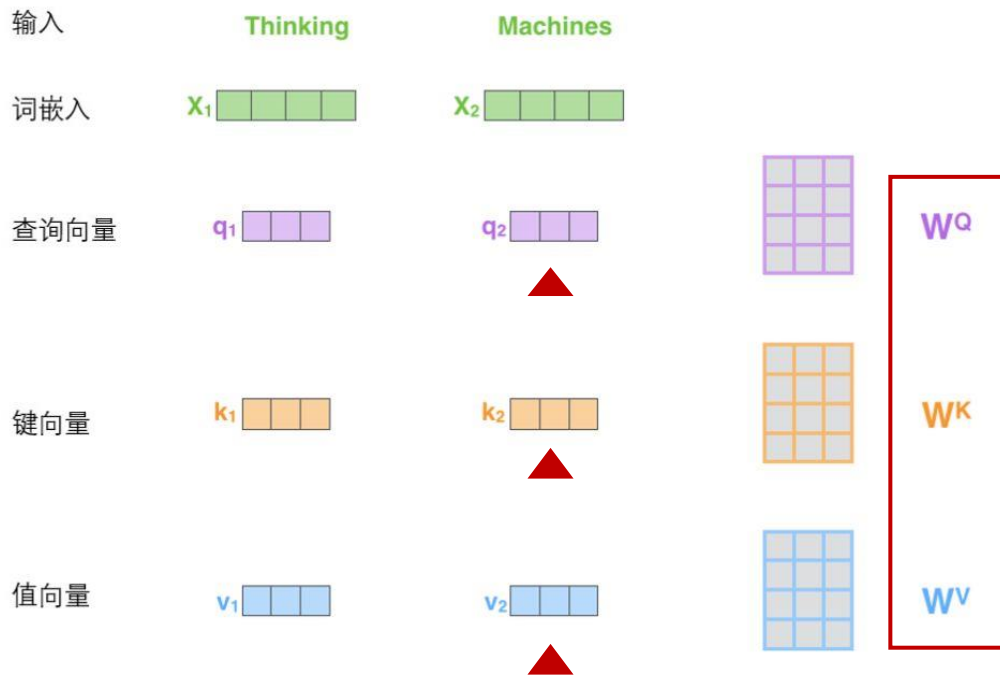
这个“it”在这个句子是指什么呢？它指的是street还是这个animal呢？

当模型处理这个单词“it”的时候，自注意力机制会允许“it”与“animal”建立联系。随着模型处理输入序列的每个单词，**自注意力**会关注整个输入序列的所有单词，帮助模型对本单词更好地进行编码。





# 自注意力机制



第一步：从每个编码器的输入向量（每个单词的词向量）中生成三个向量。对于每个单词，设置一个查询向量 $Q$ 、一个键向量 $K$ 和一个值向量 $V$ 。这三个向量是通过词嵌入与三个权重矩阵后相乘创建的。

第二步：计算得分。假设我们在为这个例子中的第一个词“Thinking”计算自注意力向量，需要拿输入句子中的每个单词对“Thinking”打分。这些分数决定了在编码单词“Thinking”的过程中有多重视句子的其它部分。

# 权值计算

输入

词嵌入

查询向量

键向量

值向量

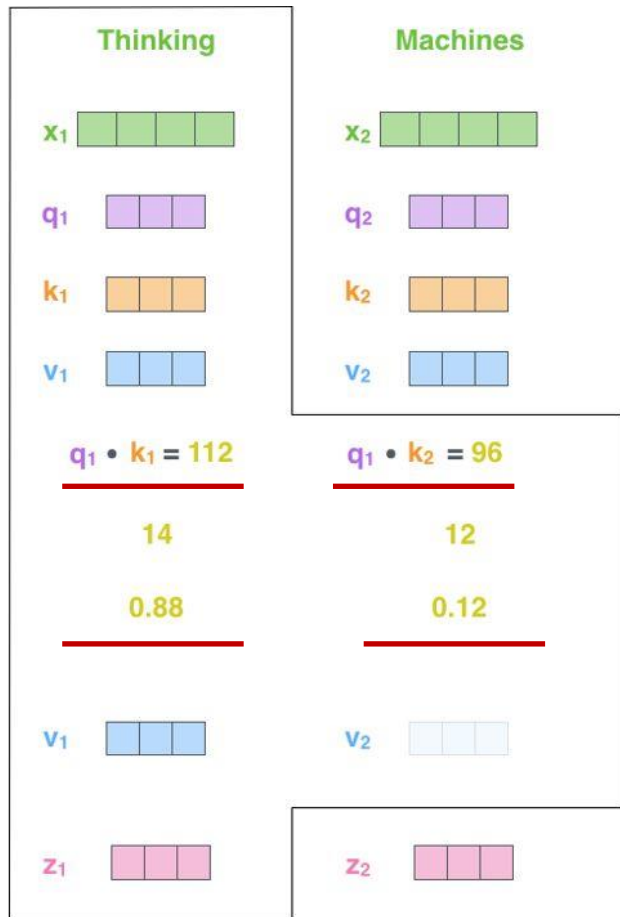
★ 打分

除以8 ( $\sqrt{d_k}$ )

Softmax

softmax  
乘以  
值向量

求和



通过打分单词（所有输入句子的单词）的键向量K与“Thinking”的查询向量相点积来计算的。如果处理位置最靠前的词的自注意力，第一个分数是 $q_1$ 和 $k_1$ 的点积，第二个分数是 $q_1$ 和 $k_2$ 的点积。（计算所有单词的权值）

第三步和第四步：将分数除以8（经验默认值），然后通过softmax传递结果。softmax的作用是使所有单词的分数归一化（百分数），得到的分数都是正值且和为1。

第五步：将每个值向量乘以softmax分数，关注语义上相关的单词

第六步：对加权值向量求和，然后即得到自注意力层在该位置的输出

得到的向量就可以传给前馈神经网络



# 矩阵计算过程

$$\begin{matrix} \text{X} \\ \begin{matrix} \text{Green} \\ 3 \times 4 \end{matrix} \end{matrix} \times \begin{matrix} \text{W}^Q \\ \begin{matrix} \text{Purple} \\ 4 \times 4 \end{matrix} \end{matrix} = \begin{matrix} \text{Q} \\ \begin{matrix} \text{Purple} \\ 3 \times 4 \end{matrix} \end{matrix}$$

第一步是计算查询矩阵、键矩阵和值矩阵。为此，将输入句子的词嵌入装进矩阵 $\text{X}$ 中，将其乘以**训练的权重矩阵**( $\text{W}^Q$ ,  $\text{W}^K$ ,  $\text{W}^V$ ) (这三个是需要训练得到的)。

$$\begin{matrix} \text{X} \\ \begin{matrix} \text{Green} \\ 3 \times 4 \end{matrix} \end{matrix} \times \begin{matrix} \text{W}^K \\ \begin{matrix} \text{Orange} \\ 4 \times 4 \end{matrix} \end{matrix} = \begin{matrix} \text{K} \\ \begin{matrix} \text{Orange} \\ 3 \times 4 \end{matrix} \end{matrix}$$

将步骤2到步骤6合并为一个公式来计算自注意力层的输出

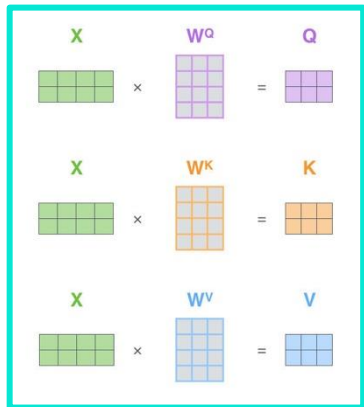
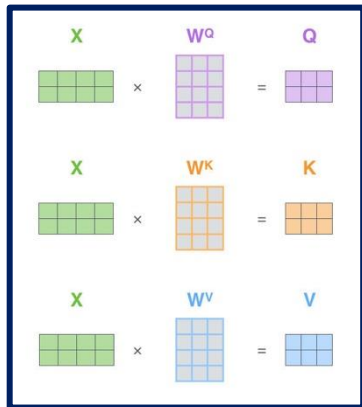
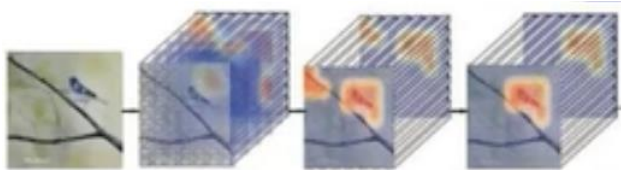
$$\begin{matrix} \text{X} \\ \begin{matrix} \text{Green} \\ 3 \times 4 \end{matrix} \end{matrix} \times \begin{matrix} \text{W}^V \\ \begin{matrix} \text{Blue} \\ 4 \times 4 \end{matrix} \end{matrix} = \begin{matrix} \text{V} \\ \begin{matrix} \text{Blue} \\ 3 \times 4 \end{matrix} \end{matrix}$$

$$\text{softmax} \left( \frac{\begin{matrix} \text{Q} \\ 3 \times 4 \end{matrix} \times \begin{matrix} \text{K}^T \\ 4 \times 3 \end{matrix}}{\sqrt{d_k}} \right) \begin{matrix} \text{V} \\ 3 \times 4 \end{matrix} = \begin{matrix} \text{Z} \\ \text{Pink} \\ 3 \times 4 \end{matrix}$$

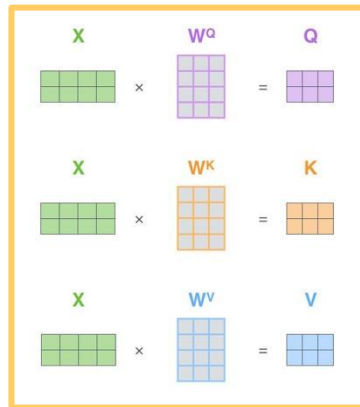
消除维度对点乘的影响

$\text{X}$ 矩阵中的每一行对应于输入句子中的一个单词

- 一组Q、K、V得到了一组当前词的特征表达
- 类似于卷积神经网络汇总的Filter，是否能提取多种特征？



...

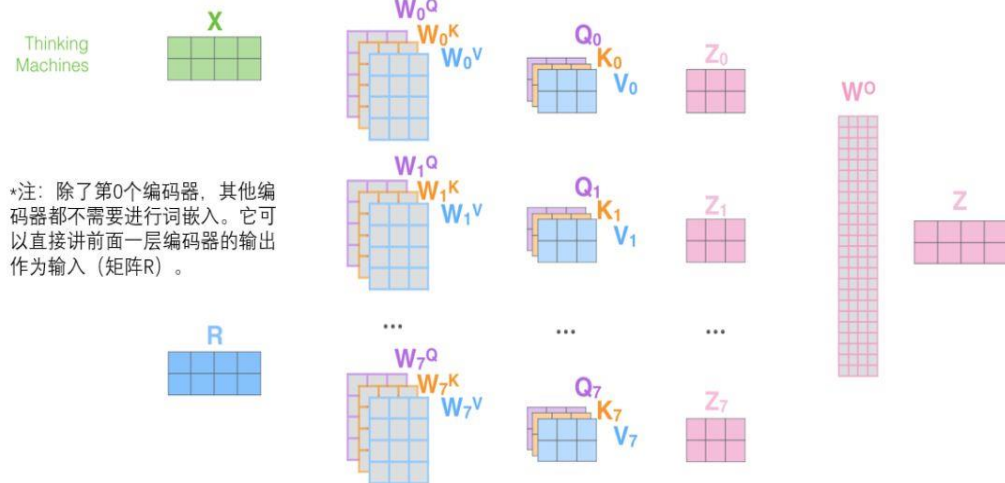


- 通过增加“多头”注意力（“multi-headed” attention）的机制，论文进一步完善了自注意力层，并在两方面提高了注意力层的性能：
- 扩展了模型专注于不同位置的能力。在上面的例子中，虽然每个编码都在 $z_1$ 中有或多或少的体现，但是它可能被实际的单词本身所支配。如果翻译一个句子，比如“The animal didn’t cross the street because it was too tired”，想知道“it”指的是哪个词，这时模型的“多头”注意机制会起到作用。
- 给出了注意力层的多个“表示子空间”（representation subspaces）。对于“多头”注意机制，有多个查询/键/值权重矩阵集（Transformer使用8个注意力头，因此对于每个编码器/解码器有8个矩阵集合）。这些集合中的每一个都是随机初始化的，在训练之后，每个集合都被用来将输入词嵌入（或来自较低编码器/解码器的向量）投影到不同的表示子空间中。

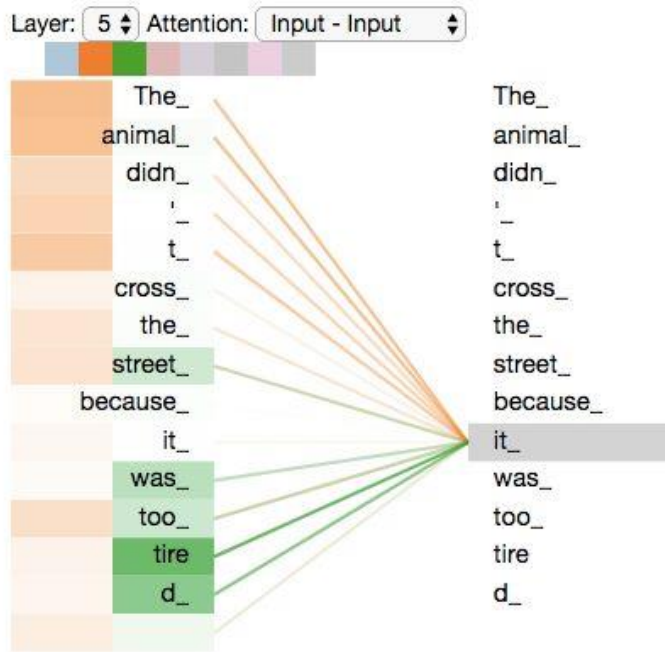


# 多头注意力示例

- 1) 这是我们的输入句子\*
- 2) 编码每一个单词
- 3) 将其分为8个头，将矩阵X或R乘以各个权重矩阵
- 4) 通过输出的查询/键/值 (Q/K/V) 矩阵计算注意力
- 5) 将所有注意力头拼接起来，乘以权重矩阵 $W^O$



通过不同的head得到多个特征，将其拼接（可能比较长，需要降维），再通过一层全连接实现降维

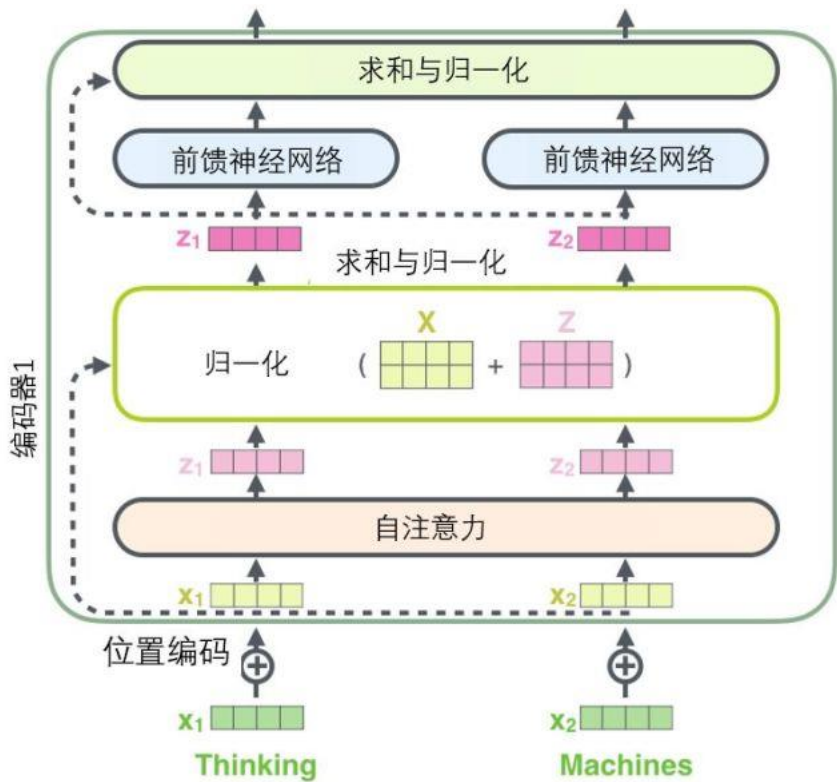


编码“it”一词时，一个注意力头集中在“animal”上，而另一个则集中在“tired”上，从某种意义上说，模型对“it”一词的表达在某种程度上是“animal”和“tired”的代表。





# 残差模块与位置编码



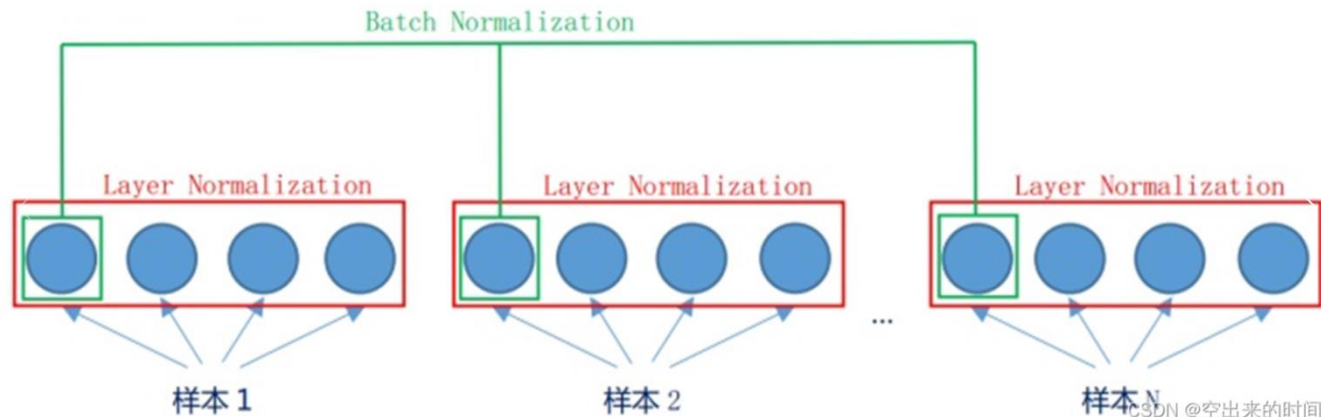
**残差模块**：在每个编码器中的每个子层（自注意力、前馈网络）的周围都有一个残差连接（图中虚线），并且都跟随着一个“层-归一化” (LN) 步骤。

序列化数据中更常使用的是**Layer Norm**，因为**序列的长度会变化**，如果使用 batch norm 的话，可能导致均值方差波动很大，从而影响效果，而 layer norm 则是**逐个样本去进行的**，不会受影响。





- BN层(Batch Normalization): 是在不同样本之间进行归一化。
- LN层(Layer Normalization): 是在同一样本内部进行归一化。



LN 是在单个样本的所有特征维度上进行归一化，不依赖 Batch 大小，非常适合处理序列数据

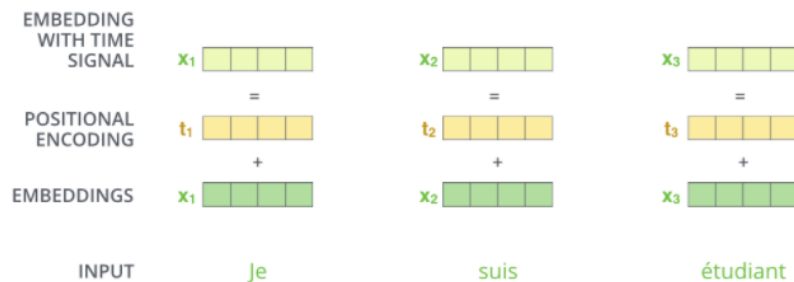
- 每个词都会考虑整个序列的加权，相当于放在哪里无所谓，但是这与实际不符，无论句子的结构怎么打乱，Transformer都会得到类似的结果
- Transformer模型并没有捕捉顺序序列的能力，需要模型能对位置有额外的输入（为了让模型知道“我爱你”和“你爱我”的区别）

位置编码（Position Embedding）：**在词向量中加入单词的位置信息**。两种方法，根据数据学习或设计编码规则：

$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d})$$

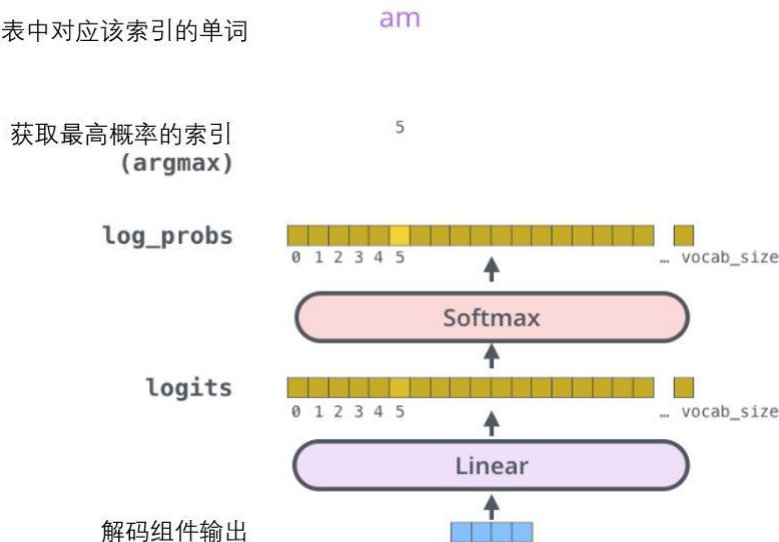
$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d})$$

其中，pos 表示单词在句子中的位置，d 表示 PE 的维度 (与词 Embedding 一样)，2i 表示偶数的维度，2i+1 表示奇数维度 (即  $2i \leq d$ ,  $2i+1 \leq d$ )。



- 解码阶段的每个步骤都会输出一个输出序列，例如英语翻译的句子中的元素
- 重复这个过程，直到到达一个特殊的终止符号，它表示Transformer的解码器已经完成了输出。
- 每个步骤的输出在下一个时间步被提供给底端解码器，类似于编码器，解码器会输出它们的解码结果。此外，可以嵌入并添加位置编码给那些解码器，表示每个单词的位置。
- 解码器中的自注意力层表现的**模式与编码器不同**：在解码器中，**自注意力层只处理输出序列中更靠前的那些位置**。在softmax步骤前，它会把后面的位置给隐去
- “**编码-解码注意力层**”工作方式基本类似多头自注意力层，是通过在它下面的层来创造查询矩阵，即**Q来自解码器的输出，从编码器的输出中取得键/值矩阵，即K和V。**

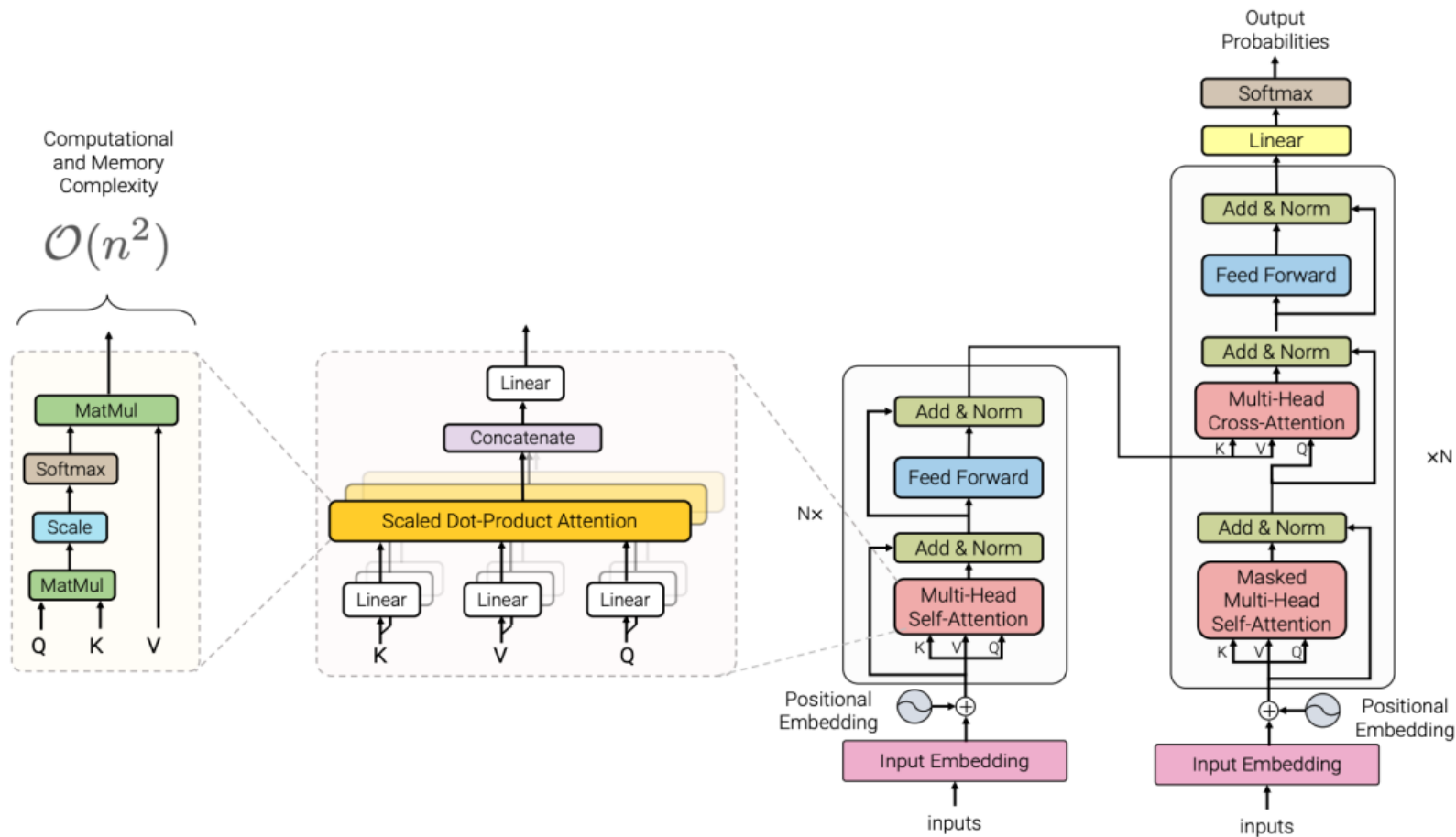
查找词表中对应该索引的单词



- 解码组件最后会输出一个实数向量，由 Softmax 层实现。
- 线性变换层是一个简单的全连接神经网络，可以把解码组件产生的向量投射到一个比它大得多的、对数几率（logits）向量里（词表大小）。例如，从训练集中学习一万个不同的英语单词，对数几率向量为一万个单元格长度的向量，每个单元格对应某一个单词的分数。
- Softmax 层将分数变成概率（都为正数、上限1.0）。概率最高的单元格被选中，并且它对应的单词被作为这个时间步的输出。



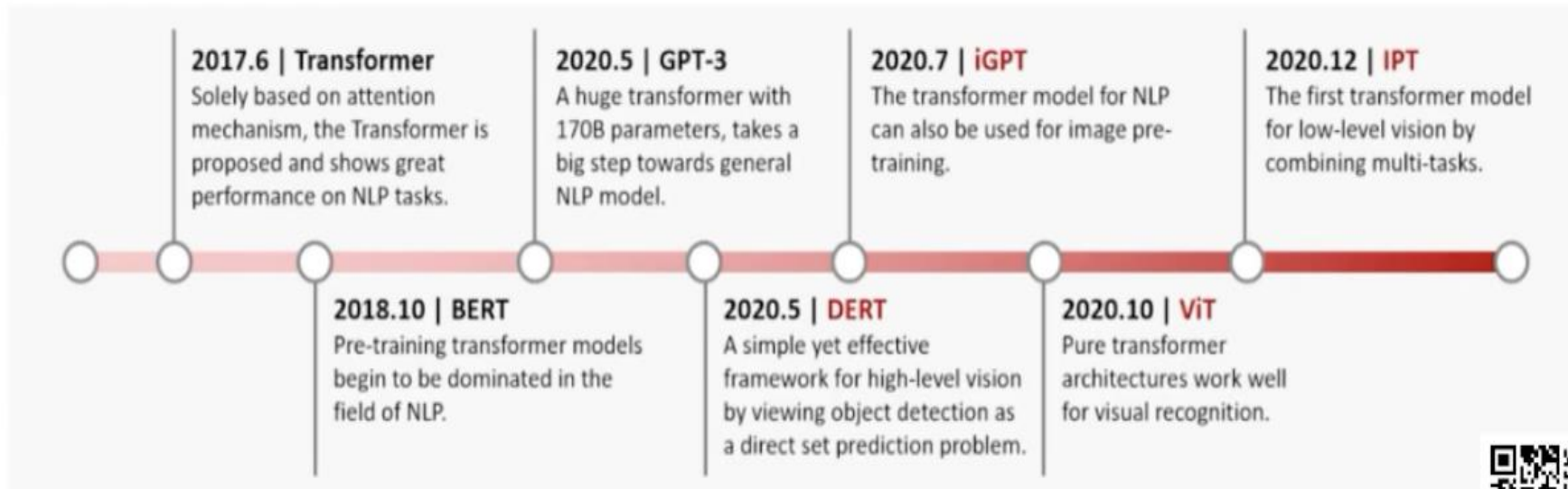
# Transformer总结



- Transformer真的火了！
- 从NLP到CV，到数据处理
- 各类大模型的基础网络！
- 卷积神经网络要被取代了么？
- 新一代的backbone，直接用于各类下游任务
- 计算机视觉的分类、分割、检测各项任务均刷榜

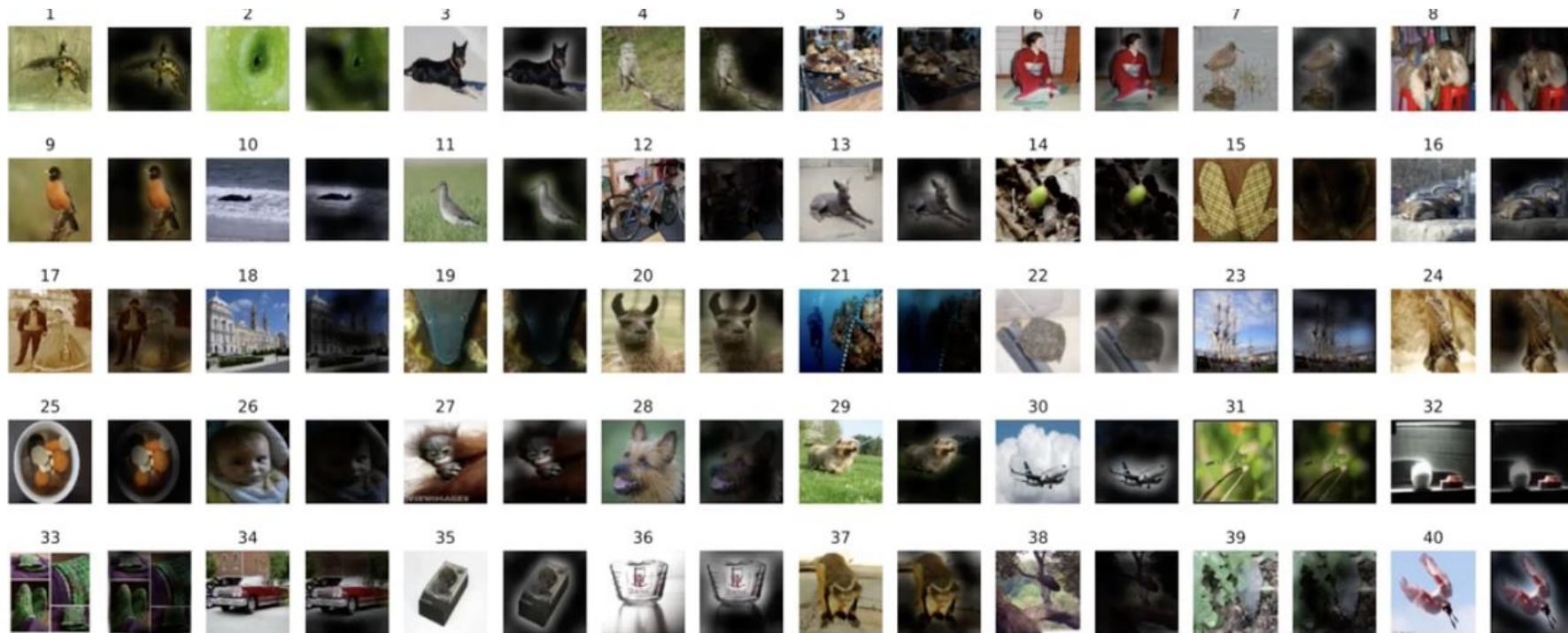


# CV中的Transformer



2017年Transformer在NLP领域爆发，20年轰动CV圈







- 动机：将Transformer直接应用于图像领域，进行图像分类任务，而不修改Transformer架构，也不使用CNN。
- 设计思路：将图像分割成大小一致的**patch**，对应于NLP中的**token**，再将其进行嵌入表示，成为Transformer的输入。在patch前加入一个class表示，对应全局信息（图片分类），进行最终的预测。

Published as a conference paper at ICLR 2021

## AN IMAGE IS WORTH 16X16 WORDS: TRANSFORMERS FOR IMAGE RECOGNITION AT SCALE

Alexey Dosovitskiy<sup>\*,†</sup>, Lucas Beyer<sup>\*</sup>, Alexander Kolesnikov<sup>\*</sup>, Dirk Weissenborn<sup>\*</sup>,  
Xiaohua Zhai<sup>\*</sup>, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer,  
Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, Neil Houlsby<sup>\*,†</sup>

<sup>\*</sup>equal technical contribution, <sup>†</sup>equal advising

Google Research, Brain Team

{adosovitskiy, neilhoulshby}@google.com

- NLP 中句子是一维的，而图像数据是二维的，如何把二维的图像数据转成跟 NLP 一样一维的？
- 按像素展开，每个像素就是一个patch，patch数 =  $224 \times 224 = 50176$ （BERT 的 patch数才512），显然不行；
- 用特征图作为 Transformer 的输入，例如先通过CNN得到 $14 \times 14$  的特征图，即 patch数 =  $14 \times 14 = 196$ ，再输入 Transformer；
- 按轴展开，这种是做了两次的自注意力，一次是横轴的自注意力，另一次是纵轴的自注意力，把  $H \times W$  的复杂度拆成了  $H + W$  的复杂度；
- 把窗口块作为一个 patch，类似卷积；



# AN IMAGE IS WORTH 16X16 WORDS: TRANSFORMERS FOR IMAGE RECOGNITION AT SCALE

Alexey Dosovitskiy<sup>\*,†</sup>, Lucas Beyer<sup>\*</sup>, Alexander Kolesnikov<sup>\*</sup>, Dirk Weissenborn<sup>\*</sup>,  
Xiaohua Zhai<sup>\*</sup>, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer,  
Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, Neil Houlsby<sup>\*,†</sup>

<sup>\*</sup>equal technical contribution, <sup>†</sup>equal advising

Google Research, Brain Team

{adosovitskiy, neilhoulby}@google.com

ICLR 2021

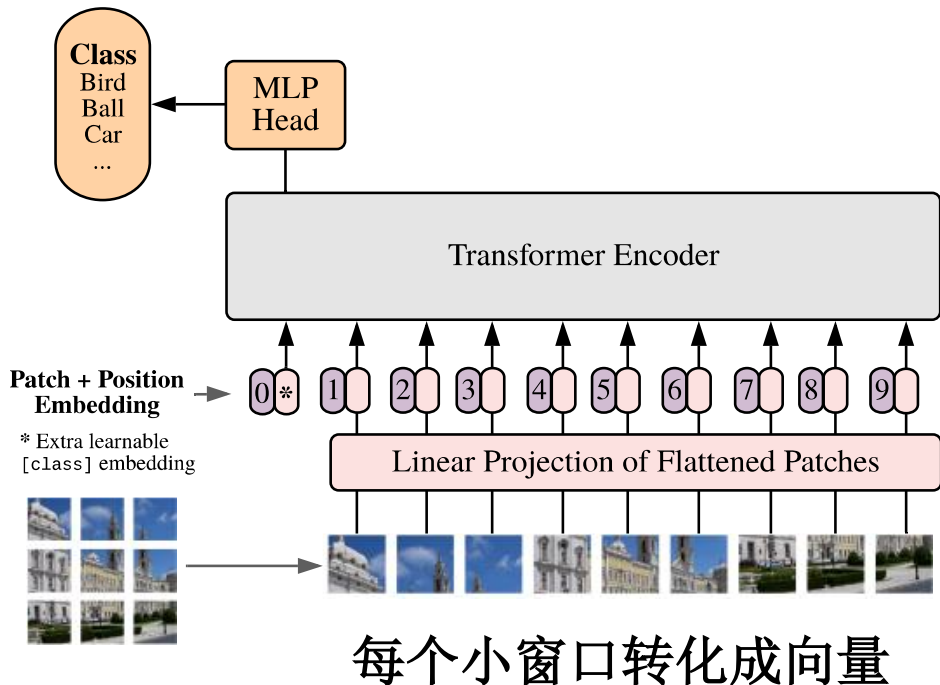
## ABSTRACT

While the Transformer architecture has become the de-facto standard for natural language processing tasks, its applications to computer vision remain limited. In vision, attention is either applied in conjunction with convolutional networks, or used to replace certain components of convolutional networks while keeping their overall structure in place. We show that this reliance on CNNs is not necessary and a pure transformer applied directly to sequences of image patches can perform very well on image classification tasks. When pre-trained on large amounts of data and transferred to multiple mid-sized or small image recognition benchmarks (ImageNet, CIFAR-100, VTAB, etc.), Vision Transformer (ViT) attains excellent results compared to state-of-the-art convolutional networks while requiring substantially fewer computational resources to train.<sup>1</sup>

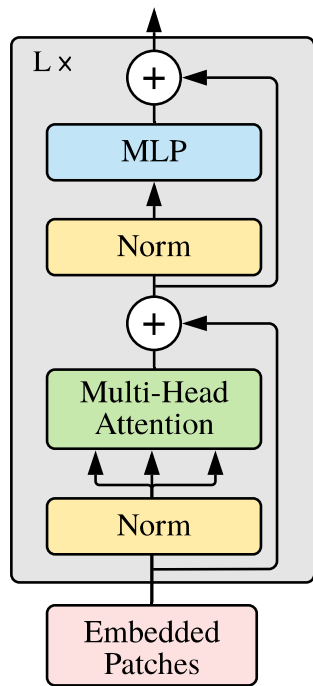


将图像切分为块，类似NLP中的词，输入Transformer网络结构中

Vision Transformer (ViT)



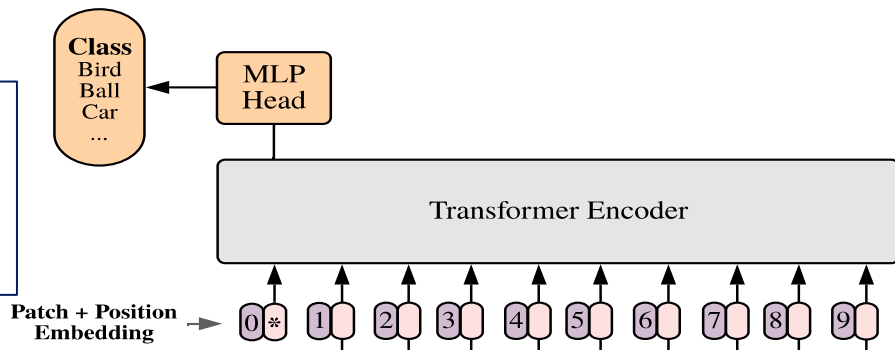
Transformer Encoder



## 位置编码

- 先在图像块后面接一个全连接层，将图像数据转换为张量数据，然后将位置编码嵌入用于表达图像块在原图的位置信息
- 一维、二维或相对位置编码的方法对结果影响不大

最前端加入一个类别编码，即 class embedding，类别编码用于最后的类别输出，参考至 BERT 的 class token

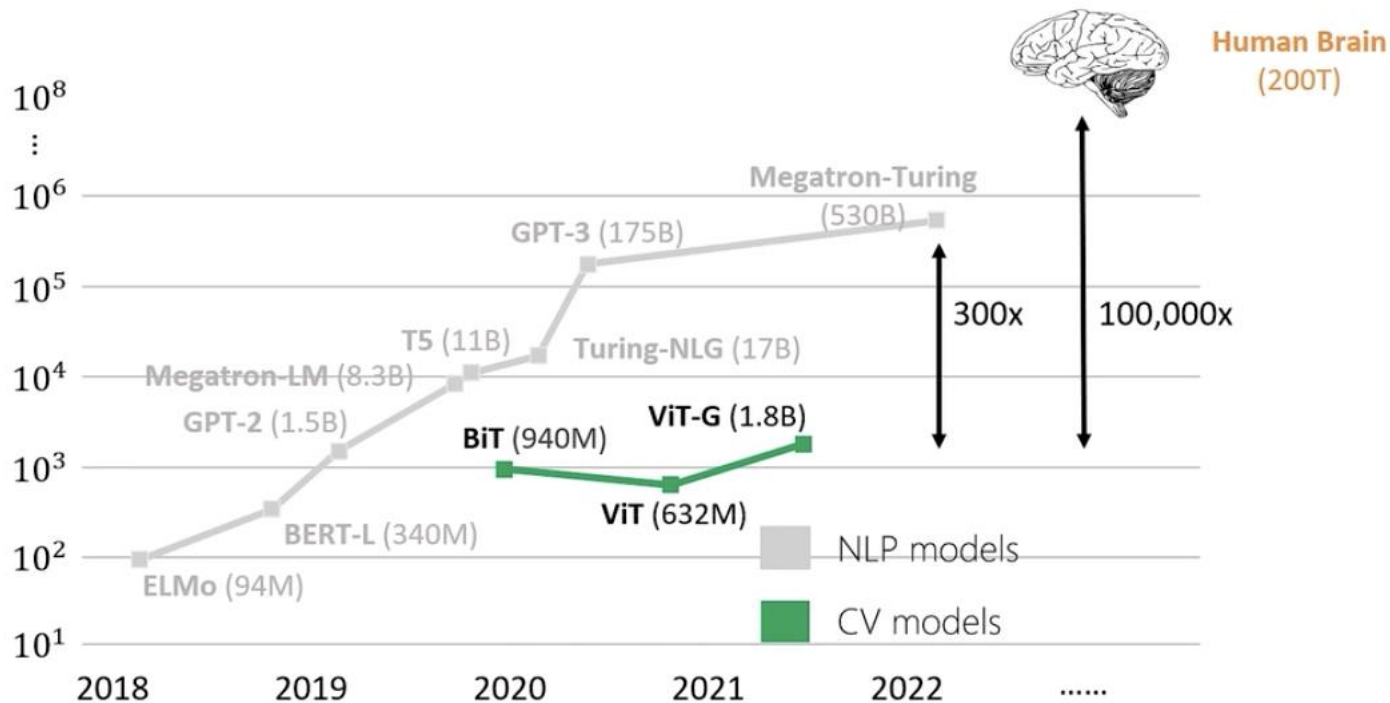


# CNN与Transformer比较

- CNN：小卷积的优势使其需要通过堆叠很多层卷积获得更大的感受野（全局信息）。
- Transformer不需要堆叠，直接获得全局信息，但需要更多的训练数据



# 各类模型的参数变化



- Swin Transformer, CVPR 2021 最佳论文
- 分类、分割、检测等任务效果提升显著，成为新一代的backbone网络，可直接用于下游任务
- 开源代码，且提供大、中、小版本可选，提供了CV领域的**预训练模型**，对标ResNet
- Swin Transformer解决了哪些问题？
  - 图像像素点太多，则需要更多特征构建长序列，但长序列计算注意力机制存在效率问题
  - 图像的感受野如何体现？

## Swin Transformer: Hierarchical Vision Transformer using Shifted Windows

Ze Liu<sup>†\*</sup> Yutong Lin<sup>†\*</sup> Yue Cao<sup>\*</sup> Han Hu<sup>\*‡</sup> Yixuan Wei<sup>†</sup>  
Zheng Zhang Stephen Lin Baining Guo  
Microsoft Research Asia

{v-zeliu1, v-yutlin, yuecao, hanhu, v-yixwe, zhez, stevelin, bainguo}@microsoft.com



## 核心思想：引入窗口与分层的方式代替长序列

- 层级结构，抽取不同层次的视觉特征，分辨率逐渐降低的过程（Transformer本身是长序列不变）
- 小窗口内进行Transformer，而非整张图

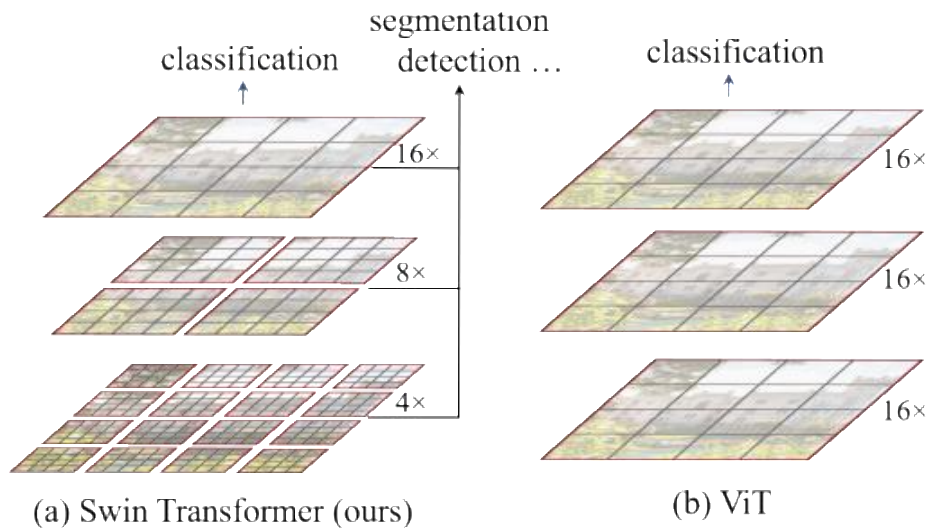


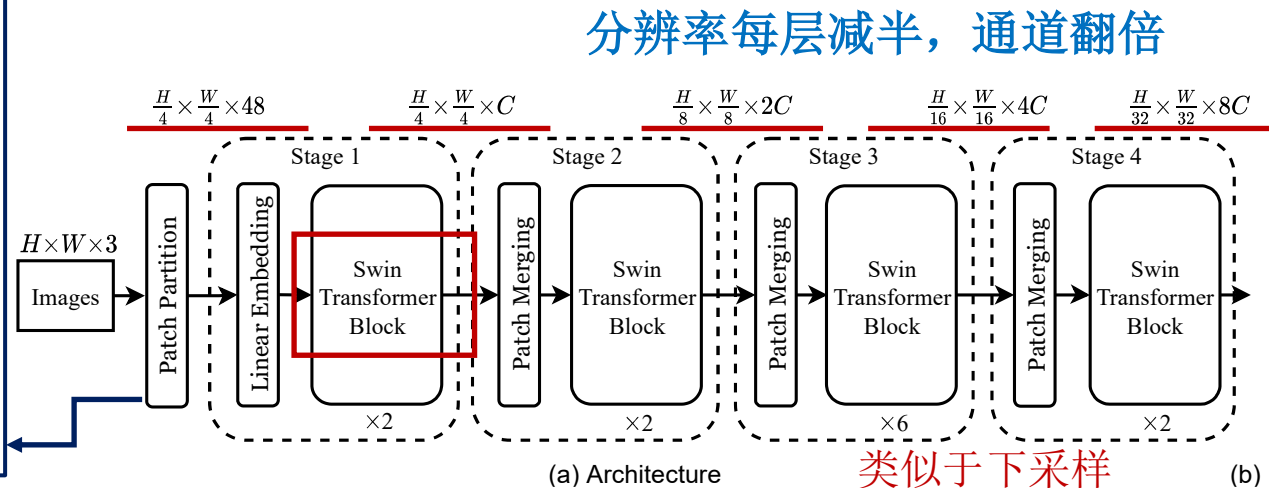
Figure 1. (a) The proposed Swin Transformer builds hierarchical

## 整体网络结构

- 由卷积提取图像表征向量，再通过向量构建Patch特征序列
- 分层计算**Attention**，核心为block，改进了Attention计算

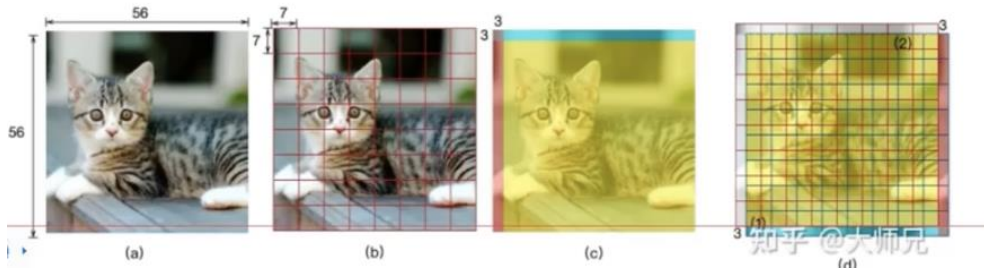
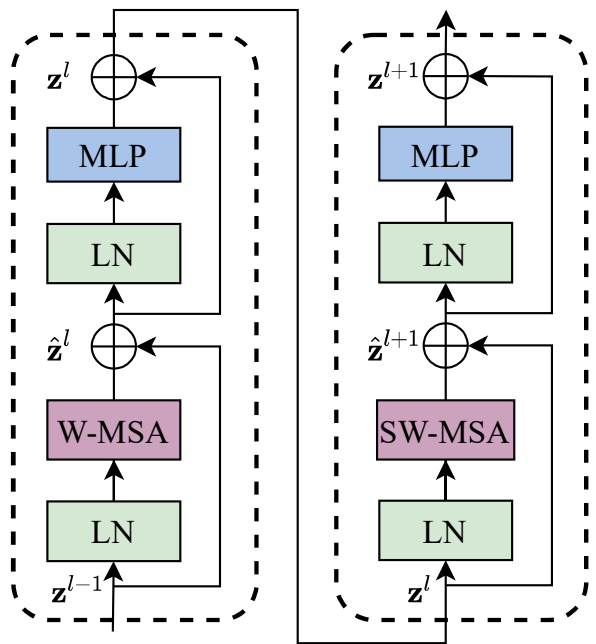
例如：

- 输入 $224 \times 224 \times 3$ 图像；
- 输出 $(3136, 96)$ ，相当于序列长度3136个，每个向量是96维特征；
- Patch Embedding的卷积核 $4 \times 4 \times 96$ ，stride=4

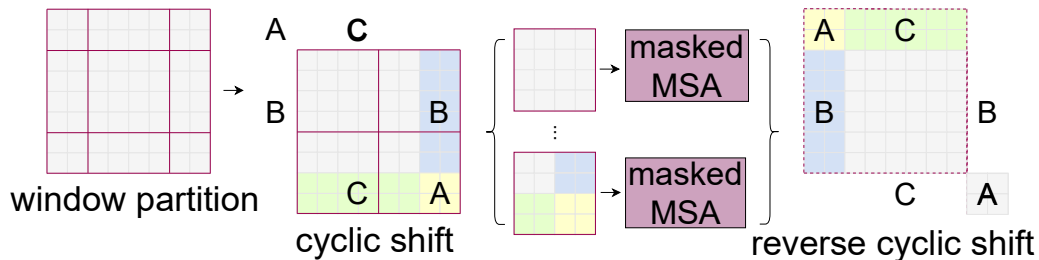


# Swin Transformer

## Swin Transformer Block



- W-MSA: 在一个小窗口内进行Attention计算，关注局部特征，降低复杂度
- SW-MSA: 窗口之间滑动，重新计算Attention，需要解决滑动后窗口个数变化的问题



滑动过程

两个模块串联，形成一个**block**

- YoLo v12采用**A<sup>2</sup>-Backbone**（**Attention-Centric Backbone**，以注意力为中心的骨干网络），重要转折点，核心突破：在骨干网络中深度集成了注意力机制，同时保持了 CNN 级别的推理速度。
- 采用了 Area Attention，将特征图划分为多个局部区域进行注意力计算，显著降低了复杂度。既获得了 Transformer 的全局感受野（能看清大物体和背景的关系），又保留了 CNN 处理局部细节的能力。

- 发展趋势：更快、更长、更省
- LLM中，传统的自注意力机制遇到了巨大的挑战：计算复杂度和内存占用随序列长度  $n$  呈平方级  $O(n^2)$  增长
- 优化 KV 缓存、实现线性计算复杂度
- 混合注意力机制（Qwen3.5）：不再让每个 token 都关注所有其他 token，而是根据任务需求，将不同的注意力模式组合起来，从而在保持性能的同时大幅提高效率。例如，如何根据内容的重要性，动态地决定哪些部分需要精细的“稠密计算”，哪些部分可以简单地“稀疏处理”，以此来节省算力

## ■ KV Cache压缩



### DeepSeek-V4: Towards Highly Efficient Million-Token Context Intelligence

DeepSeek-AI  
research@deepseek.com

#### Abstract

We present a preview version of DeepSeek-V4 series, including two strong Mixture-of-Experts (MoE) language models — DeepSeek-V4-Pro with 1.6T parameters (49B activated) and DeepSeek-V4-Flash with 284B parameters (13B activated) — both supporting a context length of one million tokens. DeepSeek-V4 series incorporate several key upgrades in architecture and optimization: (1) a hybrid attention architecture that combines Compressed Sparse Attention (CSA) and Heavily Compressed Attention (HCA) to improve long-context efficiency; (2) Manifold-Constrained Hyper-Connections (*mHC*) that enhance conventional residual connections; (3) and the Muon optimizer for faster convergence and greater training stability. We pre-train both models on more than 32T diverse and high-quality tokens, followed by a comprehensive post-training pipeline that unlocks and further enhances their capabilities. DeepSeek-V4-Pro-Max, the maximum reasoning effort mode of DeepSeek-V4-Pro, redefines the state-of-the-art for open models, outperforming its predecessors in core tasks. Meanwhile, DeepSeek-V4 series are highly efficient in long-context scenarios. In the one-million-token context setting, DeepSeek-V4-Pro requires only 27% of single-token inference FLOPs and 10% of KV cache compared with DeepSeek-V3.2. This enables us to routinely support one-million-token contexts, thereby making long-horizon tasks and further test-time scaling more feasible. The model checkpoints are available at <https://huggingface.co/collections/deepseek-ai/deepseek-v4>.